How Good Are Learned Cardinality Estimators, Really? A Comprehensive Experimental Study: [Experiments & Analysis] (Full Version)

CHUNSHAN ZHAO, Peking University, China
LINGLIN YANG, Peking University, China
LEI ZOU*, Peking University, China
XIANGYANG GOU, Peking University, China

Cardinality estimation has long been a cornerstone of query optimization. Recent advances in learned cardinality estimation (LCE) have demonstrated substantial improvements in estimation accuracy and, in some cases, end-to-end query performance compared to traditional estimators. However, whether such learned estimators can be effectively deployed in real-world DBMSs remains an open question, as practical concerns extend beyond estimation accuracy and end-to-end query latency.

In this paper, we present a systematic experimental evaluation of ten representative LCE methods across diverse datasets. We first evaluate their overall effectiveness in terms of estimation accuracy, plan quality, and end-to-end runtime. We then examine five practical aspects that are critical for real-world adoption and have not been systematically examined together in prior work, including model overhead, training data efficiency, generalization under query workload drift, scalability, and adaptability to dynamic updates. Our study finds that while LCE methods exhibit significant potential and can improve estimation accuracy and end-to-end performance in some cases, consistently outperforming PostgreSQL across diverse workloads remains challenging, particularly for large-scale databases and complex queries, and no single approach dominates across all evaluated dimensions. These findings reveal several practical limitations of current LCE techniques that may limit their practical adoption, while also highlighting promising directions for future research. We believe that our comprehensive evaluation offers valuable insights for both researchers and practitioners in the design and deployment of learned cardinality estimators.

CCS Concepts: • **Information systems → Query optimization**.

Additional Key Words and Phrases: Cardinality Estimation, Query Optimization

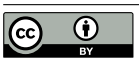
ACM Reference Format:
Chunshan Zhao, Linglin Yang, Lei Zou, and Xiangyang Gou. 2026. How Good Are Learned Cardinality Estimators, Really? A Comprehensive Experimental Study: [Experiments & Analysis] (Full Version). *Proc. ACM Manag. Data* 4, 6 (SIGMOD), Article 357 (December 2026), 35 pages. https://doi.org/10.1145/3856352

1 Introduction

Query optimization is a cornerstone of modern relational database systems [36, 75]. A fundamental component of this process is cardinality estimation (CE), which predicts the intermediate result sizes of SQL queries before execution [75]. Given an input query, traditional cost-based query optimizers explore a space of candidate execution plans and select the one with the lowest estimated

*Corresponding author.

Authors' Contact Information: Chunshan Zhao, Peking University, Beijing, China, zhaochunshan@pku.edu.cn; Linglin Yang, Peking University, Beijing, China, linglinyang@stu.pku.edu.cn; Lei Zou, Peking University, Beijing, China, zoulei@pku.edu.cn; Xiangyang Gou, Peking University, Beijing, China, gxy1995@pku.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2836-6573/2026/12-ART357
https://doi.org/10.1145/3856352

cost, where the cost model relies heavily on estimated cardinalities [18, 19, 36]. Accurate cardinality estimates are thus essential for high-quality execution plans [36, 37, 39].

Over the past decades, classical histogram-based cardinality estimation methods [10, 28, 58, 69] as well as sampling-based approaches [38, 40, 109] have been widely adopted in both open-source and commercial DBMSs [50, 59, 72]. However, these methods typically rely on simplified assumptions, such as column independence or attribute uniformity, which rarely hold in real-world data [36]. As a result, they can incur significant estimation errors and lead to suboptimal execution plans whose runtime may be orders of magnitude worse than that of the optimal plan [36, 56].

To overcome these limitations, learned approaches in query optimization have recently emerged as a promising direction [11, 111]. Using machine learning (ML) techniques, these methods can capture complex data distributions and query-data interactions without relying on strong simplifying assumptions. They can be broadly categorized into three paradigms [24, 111]:

- *Learned Cardinality Estimation (LCE)* [26, 32, 96, 102] leverages ML models to predict query cardinalities. These methods aim to replace the traditional cardinality estimators in query optimizers, while still relying on traditional cost models and plan enumeration procedures to derive the optimal plan.
- *Learned Cost Model (LCM)* [24, 25, 46, 79, 108] learns a direct mapping from a query plan to its execution cost and typically uses a traditional plan enumerator to determine the optimal plan. Cardinality estimates may be either entirely bypassed [79] or incorporated as auxiliary input features [25, 46, 54].
- *Learned Query Optimization (LQO)* [35] aims to generate execution plans either in a direct end-to-end fashion [7, 53, 95, 101], or by guiding the traditional optimizer in an indirect manner [52, 78, 100, 110], e.g., through query hint generation [52, 78, 100].

Among these, LCE is the focus of this paper for two main reasons. First, inaccurate cardinality estimates are widely recognized as the primary cause of suboptimal query plans [36, 56], and accurate cardinality estimates alone are often sufficient to select near-optimal plans even with traditional cost models [24, 36]. Second, LCE is closely related to the other two paradigms as both LCM and LQO approaches may rely on cardinality estimates as input features [25, 46, 52–54]. Studying LCE systematically can therefore provide insights not only for improving cardinality estimation itself, but also for the design and evaluation of LCM and LQO methods [24, 35].

Despite the impressive results reported in recent literature, LCE methods have seen limited adoption in real-world systems [89]. Although ML models offer greater expressive power, they also introduce several practical concerns when compared with the lightweight and interpretable traditional estimators used in DBMSs [21, 30, 89]. We summarize the main concerns for the practical adoption of LCE as follows: (1) additional time and space overheads for model training and deployment; (2) extra effort for training data collection, as some methods require queries together with their true cardinalities for supervision; (3) limited generalization capability of ML models; (4) scalability issues when handling large or complex real-world data and query workloads; and (5) difficulties in supporting dynamic updates, since maintaining model freshness in evolving databases is non-trivial. Given these challenges, LCE methods remain largely undeployed in production settings. Therefore, it is essential to develop a deeper and more systematic understanding of learned cardinality estimators so as to guide future research and enable their effective deployment in real-world DBMSs.

To this end, we present a comprehensive experimental study that evaluates both the *overall performance* and the *practical aspects* of learned cardinality estimators, aiming to shed light on the question: *"How good are learned cardinality estimators, really?"* We first examine their overall effectiveness from a query optimization perspective, including estimation accuracy, the quality

of the generated execution plans, and the resulting end-to-end query execution latency. Beyond overall performance, we further investigate a broad range of practical dimensions, including model construction and storage cost, training data efficiency, generalization under query workload drift, scalability, and adaptability to dynamic updates.

Our work differs from prior experimental studies [21, 30, 64, 73, 80, 89] in several respects: (1) we evaluate multi-table join cardinality estimation beyond the single-table settings considered in [73, 89]; (2) we consider both static and dynamic database settings, extending the static settings studied in [30, 64, 73]; (3) we investigate plan ranking and selection quality as well as end-to-end runtime in addition to estimation accuracy [73, 80, 89]; (4) we examine more practical aspects, such as training data efficiency, generalization under workload drift, and scalability; and (5) we include several recent approaches [31, 42, 62, 74, 92, 102] that have not been considered in prior experimental studies.

In summary, we make the following contributions in this paper.

- We present a taxonomy and a unified workflow for LCE, which organize the design space of existing approaches, and provide a systematic review of ten representative LCE methods.
- We conduct a comprehensive, multi-dimensional experimental evaluation of LCE approaches on three real-world datasets, covering both overall effectiveness for query optimization and practical considerations for system deployment.
- We present a systematic study of LCE methods under dynamic updates, identify correctness limitations in the update strategy of correlated-grouping methods, and propose two practically feasible alternatives.
- Through extensive empirical analysis, we identify a set of key empirical observations (O1–O14) that reveal the strengths and limitations of current LCE techniques, which further highlight several promising directions for future research.
- To ensure reproducibility and facilitate future research, we release all source code, datasets, and experimental results as open-source artifacts.[1]

2 Learned Cardinality Estimation

In this section, we first formulate the cardinality estimation problem. Next, we introduce the general workflow of learned cardinality estimators. Finally, we present a taxonomy of LCE approaches and review representative methods.

2.1 Problem Definition

We focus on selection–projection–join (SPJ) queries, which are among the most common in practice and serve as fundamental building blocks for more complex queries [3, 5, 75, 82]. Consequently, SPJ queries have become the primary target of most existing learned cardinality estimation (LCE) methods [26, 31, 32, 42, 62, 74, 92, 96, 102]. For completeness, we briefly restate the definition of the cardinality estimation problem, following prior work [30, 31, 42, 102].

Definition 2.1 (Database). A database D is a triple $(\mathcal{T}_D, \mathcal{J}_D, \mathcal{A}_D)$, where $\mathcal{T}_D$, $\mathcal{J}_D$, and $\mathcal{A}_D$ denote the sets of tables, primary-key–foreign-key (PK–FK) relationships, and attributes, respectively. Each table $T \in \mathcal{T}_D$ has a set of attributes $\mathcal{A}_T \subseteq \mathcal{A}_D$.

Definition 2.2 (Query). A query Q defined over a database D is a quadruple $(\mathcal{T}_Q, \mathcal{J}_Q, \mathcal{A}_Q, \mathcal{F}_Q)$, where $\mathcal{T}_Q \subseteq \mathcal{T}_D$ is the set of tables involved in the query, $\mathcal{J}_Q \subseteq \mathcal{J}_D$ is the set of inner join predicates, $\mathcal{A}_Q$ is the set of attributes appearing in $\mathcal{T}_Q$, i.e., $\mathcal{A}_Q = \cup_{T \in \mathcal{T}_Q} \mathcal{A}_T$, and $\mathcal{F}_Q$ is the set of selection predicates.

[1] https://github.com/ZhaoChunshan/LearnedCardinalityEstimators

Definition 2.3 (Cardinality Estimation). Given a database $D = (\mathcal{T}_D, \mathcal{J}_D, \mathcal{A}_D)$ and a query $Q = (\mathcal{T}_Q, \mathcal{J}_Q, \mathcal{A}_Q, \mathcal{F}_Q)$, the cardinality estimation problem is to estimate the size of the result obtained by executing Q on D.

We denote the true cardinality and estimated cardinality by $card(Q)$ and $\widehat{card}(Q)$, respectively. For an attribute $A \in \mathcal{A}_D$, we use $dom(A)$ to denote its domain, i.e., the set of all distinct values of A. Moreover, following prior work [31, 42, 92, 96, 102], we focus on selection predicates whose conditions on different attributes are independent, e.g., $1 \leq A \leq 4$ is allowed, whereas $A+B < 4$ is not. We use $rng_Q(A)$ to denote the selection range of query Q on attribute A, i.e., the subset of $dom(A)$ that satisfies the predicates in $\mathcal{F}_Q$. If no selection predicate is applied to A, then $rng_Q(A) = dom(A)$. When the context is clear, we omit the subscript Q in $rng_Q(A)$ for brevity.

2.2 Workflow of Learned Cardinality Estimators

Following prior studies [26, 89, 96, 97], learned cardinality estimators can be broadly categorized into two major paradigms: *query-driven* and *data-driven*. Figure 1 illustrates their overall workflow.

Query-driven methods [30, 42, 62, 74, 102] train ML models to directly learn a mapping from query features to their cardinalities in a *supervised* fashion. At the training stage, these methods require a set of training queries with ground-truth cardinalities, which can be obtained from query logs or by executing a representative workload. Training queries are first encoded into feature vectors along with database statistics (e.g., histograms), optionally incorporating estimates from classical database (DB) cardinality estimators [62, 74, 102]. These features are then used to train a regression model with ground-truth cardinalities as labels. During inference, a query Q is featurized into feature vectors and fed into the trained model to produce an estimated cardinality $\widehat{card}(Q)$.

The query-driven paradigm provides a clear problem formulation and allows the use of off-the-shelf regression models. These methods are typically lightweight and efficient at inference time. However, their effectiveness relies on the availability of a large number of high-quality queries with ground-truth cardinality labels, whose collection is often expensive and time-consuming [74, 89].

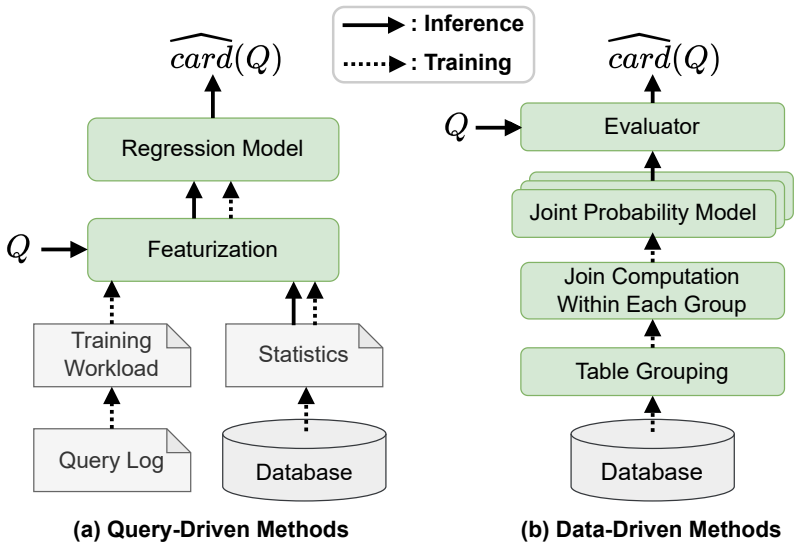


Fig. 1. Workflows of (a) query-driven methods and (b) data-driven methods.

Table 1. Taxonomy of query-driven learned cardinality estimators.

Estimator	Query Representation	Data Features	DB Cardinality Estimates	Model Architecture	Update Support
MSCN [32]	Set	Sample bitmaps	None	Multi-Set Convolutional Network	✗
RobustMSCN [62]	Set	Join bitmaps	Table Selectivity + Query Cardinality	Multi-Set Convolutional Network	✓
JGMP [74]	Graph	Sample bitmaps	Table Selectivity	Message-Passing Neural Network	✗
ALECE [42]	Vector	Histograms	None	Transformer	✓
PRICE [102]	Set	Histograms + Join scaling factors	Table Selectivity + Query Cardinality	Multi-Head Self-Attention	✓

Table 2. Taxonomy of data-driven learned cardinality estimators.

Estimator	Table Grouping	Training Data	Model Architecture	Inference Procedure	Update Strategy
DeepDB [26]	Correlated	Full outer join of each group	Sum-Product Network	Bottom-Up Traversal	Incremental / Refitting
NeuroCard [96]	Universal	Full outer join of all tables	Deep Autoregressive Model	Importance Sampling	Fine-tuning
BayesCard [93]	Correlated	Full outer join of each group	Bayesian Network	Variable Elimination	Incremental / Refitting
FactorJoin [92]	Singleton	Each individual table	Bayesian Network	Variable Elimination	Incremental
ASM [31]	Singleton	Each individual table	Deep Autoregressive Model	Importance Sampling	Fine-tuning

In contrast, data-driven methods [31, 92, 93, 96, 97] adopt an *unsupervised* paradigm, modeling the underlying joint data distribution and leveraging it for cardinality estimation. Their training pipeline typically consists of three stages: table grouping, join computation within each group, and model training. First, the set of tables $\mathcal{T}_D$ is organized into a collection of groups $\mathcal{G} = \{G_1, \ldots, G_{|\mathcal{G}|}\}$, where each group G contains a subset of tables $\mathcal{T}_G$ along with their join relationships $\mathcal{J}_G$. Each table $T \in \mathcal{T}_D$ is required to appear in at least one group. Next, for each group, full outer joins are computed, and tuples of the resulting joined tables serve as the training data. These tuples can be obtained either via explicit materialization [26] or via sampling [96]. Specifically, when a group contains only a single table, tuples from that table are used directly as training data. Finally, for each group, a separate model is trained on the full outer join to capture the joint data distribution. During inference, an evaluator combines multiple group-level models to estimate the cardinality $\widehat{card}(Q)$ for a query Q.

While data-driven methods eliminate the need for training queries, they require modeling the underlying data distribution and face inherent trade-offs among estimation accuracy, model complexity, and training and inference efficiency, especially as database scale and query complexity grow.

2.3 Taxonomy of Query-Driven Methods

We select ten open-source LCE approaches [26, 31, 32, 42, 62, 74, 92, 93, 96, 102] that broadly cover the design space, focusing on methods that show strong performance in prior studies or are recent ones not covered by existing experimental studies [21, 30, 64, 73, 80, 89]. To systematically analyze design choices, we organize both query-driven and data-driven methods along several orthogonal dimensions. Table 1 summarizes the taxonomy of query-driven methods. The taxonomy of data-driven methods is deferred to Section 2.4.

Query Representation. The first key dimension of query-driven methods is how a query is represented and encoded into feature vectors. MSCN [32] and RobustMSCN [62] represent a query Q as three sets: tables $\mathcal{T}_Q$, joins $\mathcal{J}_Q$, and predicates $\mathcal{F}_Q$, and then encode them independently. JGMP

[74] enhances this set representation by modeling a query as a join graph, where nodes correspond to table occurrences and edges represent join relationships. In contrast, ALECE [42] encodes a query into a single flat vector, where binary indicators are used to indicate the presence of tables $\mathcal{T}_Q$ and joins $\mathcal{J}_Q$, and the selection range $rng_Q(A)$ for each attribute $A \in \mathcal{A}_D$ is represented as an interval. PRICE [102] differs from previous query-driven methods in that it is pretrained on multiple databases and does not encode DB-specific schema information. It represents a query Q as (i) attribute distributions (e.g., histograms normalized to [0,1]), (ii) normalized filtering ranges on each attribute, and (iii) scaling factors for joins, which indicate the number of matched tuples in the opposite table for a given join key value.

Data Features. To better capture underlying data distributions, query-driven methods often incorporate auxiliary data features derived from database samples or statistics. MSCN maintains a sample for each table $T \in \mathcal{T}_D$. Given a query Q, it derives a *sample bitmap* for each $T \in \mathcal{T}_Q$ based on the filtering predicates, indicating which sampled tuples satisfy the predicates. JGMP further adopts a greedy unsupervised feature selection algorithm [14] to retain only the most informative samples. In contrast, ALECE and PRICE adopt histograms as data features, with PRICE additionally incorporating join scaling factors to capture cross-table correlations.

DB Cardinality Estimates. Some query-driven methods [62, 74, 102] leverage the estimated table selectivity (i.e., the fraction of tuples satisfying query predicates), or estimated query cardinality provided by a classical DB cardinality estimator as input features. JGMP only uses the estimated selectivity for each query table to enhance the features of table occurrences $\mathcal{T}_Q$, while RobustMSCN and PRICE also adopt the estimated query cardinality.

Model Architecture. The model architecture varies across methods, mainly due to different query representations and data features. MSCN and RobustMSCN employ Multi-Set Convolutional Networks [32] to encode all sets $\mathcal{T}_Q$, $\mathcal{J}_Q$, and $\mathcal{F}_Q$. JGMP adopts a Message Passing Neural Network [17, 20, 86] over the join graph. ALECE leverages a transformer [85] architecture, where the transformer encoder processes histogram features and the transformer decoder integrates query representations. PRICE employs multiple self-attention [85] modules to jointly encode all input features.

Update Support. Query-driven estimators [32, 74] are typically tied to a static database instance and cannot handle data updates, as they are trained on query-cardinality pairs generated over a fixed snapshot. Recent works mitigate this limitation by training on workloads collected from evolving databases [42, 62] or multiple databases [102], together with enhanced data features, allowing models to capture the joint dependence between queries and database states. As a result, these methods can accommodate data updates by refreshing the database statistics, reducing or avoiding the need for model fine-tuning.

2.4 Taxonomy of Data-driven Methods

We present the taxonomy of data-driven methods in Table 2.

Table Grouping. Data-driven methods organize the set of tables $\mathcal{T}_D$ into groups and train a separate joint probabilistic model for each group. Table grouping strategies can be broadly categorized into three types [30, 31]: *correlated*, *universal*, and *singleton*. Correlated grouping methods [26, 93, 113] cluster tables based on inter-table correlations, where the correlation between two tables is typically quantified by the maximum Randomized Dependence Coefficient (RDC) [51] over all cross-table attribute pairs. Universal grouping methods [90, 96] place all tables in $\mathcal{T}_D$ into a single group and train a unified model, typically assuming an acyclic database schema. Finally, singleton grouping methods [31, 92] treat each table $T \in \mathcal{T}_D$ as an individual group and train a model per table.

Training Data. Given a table grouping, the training data is typically constructed by computing the full outer join over all tables within each group. Full outer joins are used instead of inner joins, as they retain information about unmatched tuples across participating tables. When the full outer join cannot be exhaustively materialized, existing methods [26, 96] rely on sampling approaches [109] to generate uniform samples from the join result. For singleton grouping methods, where each group contains only a single table, training tuples are directly drawn from each individual table.

To support inner join cardinality estimation, for each group G, two types of auxiliary columns are introduced into the full outer join: (1) an indicator column I_T for each table $T \in \mathcal{T}_G$, indicating whether a tuple originates from a real tuple in T or is introduced via null-padding; and (2) a fanout column $F_{T_1T_2}$ for each join $J = (T_1, T_2) \in \mathcal{J}_G$, recording the number of matching tuples in T_2 with the same join-key value for each tuple in T_1.

Model Architecture. Data-driven methods employ ML models to learn the joint probability distribution $P(A_1, A_2, \cdots, A_n)$ over the attributes of a relation T, where T can be either a single table in $\mathcal{T}_D$ or the full outer join of multiple tables. Commonly used models include probabilistic graphical models such as Sum-Product Networks (SPNs) and Bayesian Networks (BNs), as well as deep autoregressive (DAR) models.

An SPN [57, 68] represents the joint distribution via a hierarchical composition of sum and product nodes, capturing mixtures and factorizations of sub-distributions. Sum nodes partition tuples into subsets with similar characteristics, while product nodes model factorizations over subsets of attributes under conditional independence assumptions. Leaf nodes model univariate distributions over individual attributes.

A BN [16, 67, 83] represents the joint distribution as a directed acyclic graph (DAG), where each node corresponds to an attribute and edges represent direct dependencies. Each attribute A_i is associated with a conditional probability distribution $P(A_i \mid Par(A_i))$, where $Par(A_i)$ denotes the parents of A_i. In a BN, each attribute is conditionally independent of its non-descendants given its parents. Accordingly, the joint distribution factorizes as $P(A_1, A_2, \cdots, A_n) = \prod_{i=1}^{n} P(A_i \mid Par(A_i))$.

A DAR model [15, 60, 85] factorizes the joint distribution in an autoregressive manner as $P(A_1, A_2, \ldots, A_n) = \prod_{i=1}^{n} P(A_i \mid A_{j<i})$. For each attribute A_i, the model conditions on the preceding attributes $A_1, \ldots, A_{i-1}$ to estimate the conditional distribution. The probability of a tuple $t = (a_1, \ldots, a_n)$ is obtained via sequential autoregressive evaluations. The model is typically trained by minimizing the cross-entropy loss [22] over training tuples.

Inference Procedure. The inference procedure for cardinality estimation depends on both the underlying probabilistic model and the adopted table grouping strategy.

For correlated [26, 93] and universal grouping [96] methods, queries fully contained in a group G are estimated as expectations under the learned joint distribution, as shown in Equation (1). This formulation adjusts for the use of full outer joins through auxiliary columns. A detailed explanation is provided in Appendix A. Here, $\widehat{P}_G$ denotes the learned joint distribution over the full outer join of $\mathcal{T}_G$, $t[\cdot]$ extracts the value of an attribute from tuple t, and $\mathbf{1}_{t[A]\in rng(A)}$ is an indicator function. Queries spanning multiple groups are handled by assuming independence across groups [26].

$$\widehat{card}(Q) = |\bowtie G| \cdot \mathop{\mathbb{E}}_{t\sim\widehat{P}_G}\left[\frac{\prod_{A\in\mathcal{A}_Q}\mathbf{1}_{t[A]\in rng(A)}\cdot\prod_{T\in\mathcal{T}_Q}t[I_T]}{\prod_{J=(T_1,T_2)\notin\mathcal{J}_Q}t[F_{T_1T_2}]}\right]. \tag{1}$$

For SPN-based methods, DeepDB [26] computes the expectation via bottom-up traversal. For DAR-based methods, NeuroCard [96] employs an importance sampling approach, referred to as progressive sampling, to approximate the expectation. For BN-based methods, BayesCard [93] applies variable elimination to evaluate the expectation.

Singleton grouping methods [31, 92] combine per-table models to estimate join query cardinalities. FactorJoin [92] first infers the distribution of join keys conditioned on filtering predicates, and then matches join-key values across tables for cardinality estimation. To improve efficiency, it bucketizes join keys and aggregates per-bucket statistics to compute a probabilistic upper bound on the join cardinality. It further adopts a dynamic programming (DP)-based estimation procedure that computes the cardinalities of all subqueries under an independence assumption over join keys. ASM [31] employs sequential importance sampling to combine per-table DAR models, in which filtering predicates and join conditions are incorporated during sampling. It further extends this process into a DP-based procedure by reusing shared samples across subqueries.

Update Strategy. When the database is updated, data-driven methods must update their models to fit the new data distribution. DAR models do not naturally support incremental updates, and DAR-based methods [31, 96] fine-tune the models on updated data. In contrast, SPNs and BNs support incremental updates: given updated tuples, SPNs propagate updates from root to leaves and update the corresponding nodes accordingly, while BNs incrementally adjust the conditional probability distributions at each node.

For correlated grouping methods, model updates introduce an additional challenge, as they require determining *how the full outer join of each group changes under updates to individual tables*. While DeepDB provides an implementation of this process, we identify potential correctness issues and propose two update strategies, as detailed in Section 5.5. The first strategy explicitly computes the delta of the full outer join and incrementally updates the parameters of the underlying SPN or BN model. The second strategy recomputes the entire full outer join on the updated database and refits only the model parameters while keeping the learned SPN or BN structure fixed. This latter approach is motivated by the observation that, in some cases, computing the delta of the full outer join can be more expensive than recomputing the join from scratch. We evaluate both incremental and refitting strategies in Section 5.5.

3 Evaluation Plan

3.1 Evaluation Aspects

We evaluate LCE methods in terms of overall query optimization performance and five practical aspects. In addition, we explore the potential of integrating learned and traditional estimators. The evaluation aims to answer the following research questions:

- **Overall Performance.** How effective are learned estimators for query optimization in terms of estimation accuracy, plan quality, and end-to-end query execution time? (Section 4)
- **Model Construction and Storage Cost.** What are the training overhead and storage footprint of LCE methods? (Section 5.1)
- **Training Data Efficiency of Query-Driven Methods.** How much training data do query-driven methods require to achieve satisfactory estimation accuracy? (Section 5.2)
- **Generalization under Query Workload Drift.** How well do learned estimators generalize under query workload shifts? (Section 5.3)
- **Scalability.** How do learned estimators scale with increasing database size and query complexity? (Section 5.4)
- **Adaptability to Dynamic Updates.** How effectively can learned estimators adapt to data updates? (Section 5.5)
- **Integration with Traditional Methods.** Can learned estimators be effectively combined with traditional methods to achieve better cardinality estimation performance? (Section 5.6)

Table 3. Statistics of Databases and Query Workloads

Workload	Database Statistics (Data)								Workload Statistics (Query)					
	Size (MB)	# of Tables	# of Rows	# of Cols	Total Attr. Domain Size	# PK-FK Joins	# Join Keys	# Equivalent Key Groups	# of Queries	# of Tables	# Filter Cols	# of Subqueries	Cardinality Range	PostgreSQL Runtime (s)
JOB-LIGHT	3,034	6	$6.2 \cdot 10^7$	37	$8.4 \cdot 10^7$	5	6	1	70	2 – 5	1 – 4	3 – 31	$2 - 1.3 \cdot 10^{10}$	$9.4 \cdot 10^{-4} - 743.6$
JOB-M	3,101	16	$6.3 \cdot 10^7$	77	$8.7 \cdot 10^7$	15	23	8	88	2 – 11	0 – 5	3 – 544	$0 - 7.6 \cdot 10^{11}$	$1.6 \cdot 10^{-3} - 770.7$
JOB-LARGE	4,131	21	$7.4 \cdot 10^7$	108	$1.2 \cdot 10^8$	20	31	11	131	2 – 19	2 – 6	3 – 53474	$1 - 9.3 \cdot 10^{12}$	0.11 – 3888.8
STATS-CEB	32	8	$1.0 \cdot 10^6$	43	$1.9 \cdot 10^6$	12	14	2	146	2 – 7	1 – 10	3 – 82	$16 - 1.6 \cdot 10^{12}$	0.033 – 2406.3
ErgastF1	15	14	$5.5 \cdot 10^5$	98	$3.2 \cdot 10^5$	21	27	6	148	2 – 7	0 – 9	3 – 82	$1 - 2.2 \cdot 10^{10}$	0.026 – 1221.0

3.2 Evaluation Metrics

We summarize the key metrics used in our study, which capture the performance of LCE methods from different perspectives.

- **Q-Error** [56] is defined as the multiplicative error between the estimated cardinality $\widehat{card}(Q)$ and the true cardinality $card(Q)$: $\text{Q-Error}(Q) = \max\{\widehat{card}(Q)/card(Q), card(Q)/\widehat{card}(Q)\}$.
- **P-Error** [21] quantifies the impact of cardinality estimation errors on query plan quality. Let C denote the true cardinalities for all subqueries $Q_{sub} \subseteq Q$, and $\hat{C}$ the corresponding estimates. Let P^C be the execution plan generated using C, and $P^{\hat{C}}$ the plan generated using $\hat{C}$. Under the assumption of a sufficiently accurate cost model, P^C can be regarded as the optimal plan. The estimated cost of $P^{\hat{C}}$ evaluated under the true cardinalities C, denoted as $\widehat{cost}(P^{\hat{C}}, C)$, is used as a proxy for the actual execution time. P-Error is then defined as the relative cost: $\text{P-Error}(Q) = \widehat{cost}(P^{\hat{C}}, C)/\widehat{cost}(P^C, C)$.
- **Subquery Rank Correlation** measures the consistency between the estimated and ground-truth subquery rankings using Spearman's rank correlation coefficient [77].
- **Plan Rank Correlation** measures how well the estimated cardinalities preserve the ranking of candidate query plans under the cost model. Given the plan costs $\widehat{cost}(P, \hat{C})$ and $\widehat{cost}(P, C)$ under the estimated and true cardinalities, respectively, the metric is defined as Spearman's rank correlation between the two cost sequences over all candidate plans.
- **Plan Execution Time** denotes the time required to execute the query plan generated based on the estimated cardinalities.
- **End-to-End Time** represents the total query latency, covering cardinality estimation and subsequent plan execution.
- **Training Time** measures the time required for offline training.
- **Model Size** reflects the storage cost of learned estimators.

3.3 Datasets and Workloads

Dataset and Test Workload. We use three real-world datasets that are widely adopted for evaluating LCE methods: IMDb [36], STATS [9], and ErgastF1 [8], together with five benchmark query workloads on them. The statistics of the datasets and workloads are summarized in Table 3. Among the five workloads, JOB-LIGHT, STATS-CEB, and ErgastF1 are used by default, while JOB-M and JOB-LARGE are only used in scalability experiments (Section 5.4).

The IMDb dataset is a real-world Internet movie database, first used to evaluate query optimizers in prior work [36]. We evaluate three workloads on it: JOB-LIGHT [32], JOB-M [96], and JOB-LARGE [30]. We exclude the original JOB workload [36] as it uses SQL constructs unsupported by many learned estimators [26, 32, 42, 93, 96, 102].[2] For JOB-M and JOB-LARGE, we remove unsupported filtering predicates (`!=`, `(NOT) LIKE`, `(NOT) ILIKE`, `(NOT) IN`, and `IS (NOT) NULL`)

[2]This includes string matching predicates (e.g., `LIKE`, `ILIKE`), negation and containment operators (e.g., `!=`, `IN`), logical disjunction (`OR`), and repeated occurrences of the same table in a query.

so that all queries can be processed by every evaluated estimator, which enables a controlled comparison on a common query workload that can be processed by all evaluated implementations. After removing these predicates, some queries become identical and are deduplicated. In addition, we discard queries that do not finish within a reasonable time on PostgreSQL to keep the end-to-end evaluation tractable. For JOB-LARGE, we further randomly select 131 queries to keep the workload runtime manageable.

The STATS-CEB benchmark [21] is built on the STATS dataset [9], which is an anonymized dump of user-contributed content from the Stats Stack Exchange network, and exhibits a large number of attribute correlations and skewed data distributions.

The ErgastF1 dataset [8] provides extensive information on Formula 1 races from the 1950 season to the present day, and has been used in prior work [62, 102] for evaluating cardinality estimators. We adopt the test workload used in prior work [102].

Training Workload. Following existing studies [21, 32, 74, 102], we generate random training queries by selecting subgraphs from the schema graph and attaching filtering predicates. Due to space constraints, we provide the detailed query generation procedure in Appendix C. For each database, we collect 10^5 queries with true cardinalities as the training workload.

3.4 Implementation and System Settings

Comparison Methods. We compare LCE methods against three non-learned CE baselines. (1) PostgreSQL [72] is the default cardinality estimator in PostgreSQL, based on one-dimensional histograms. (2) PostgreSQL-Ext [70] additionally enables PostgreSQL's extended statistics. PostgreSQL supports extended statistics over multiple columns within the same table. We use a heuristic to identify correlated column sets for constructing extended statistics, with details provided in Appendix D. (3) LpBound [104] is a recent pessimistic CE method based on information theory.

LCE Implementations. We use official open-source implementations of all evaluated LCE methods, as referenced in our repository. Hyperparameters are set according to the original papers to ensure a consistent comparison. Since NeuroCard [96] assumes an acyclic schema, which does not hold for STATS-CEB and ErgastF1, we follow prior work [21] by partitioning the schema into multiple tree-structured join schemas and training one NeuroCard model per partition. For PRICE [102], we pretrain the model from scratch using the same configuration and training workloads as described in the original paper, which consist of queries over 26 datasets, with 5×10^4 queries per dataset and 1.3×10^6 queries in total.

System Settings. All experiments are conducted on a server with an Intel Xeon Gold 5218R CPU, two NVIDIA A40 GPUs, and 128 GB of RAM, running CentOS Linux 7. End-to-end experiments are performed on PostgreSQL 13.1, compiled with the -O2 optimization flag. We tune PostgreSQL configurations following prior work [36] for in-memory query execution, build indices on all PK and FK columns, and disable parallel execution. We adopt PilotScope [112] to inject estimated cardinalities into PostgreSQL. We use PostgreSQL 13.1 because it is the PostgreSQL version supported by PilotScope. Before executing each workload, we clear the system cache, restart PostgreSQL, and warm up all data and index pages using the pg_prewarm extension [71].

4 Overall Performance

We first evaluate the overall effectiveness of LCE methods considering estimation accuracy, plan quality, and end-to-end performance.

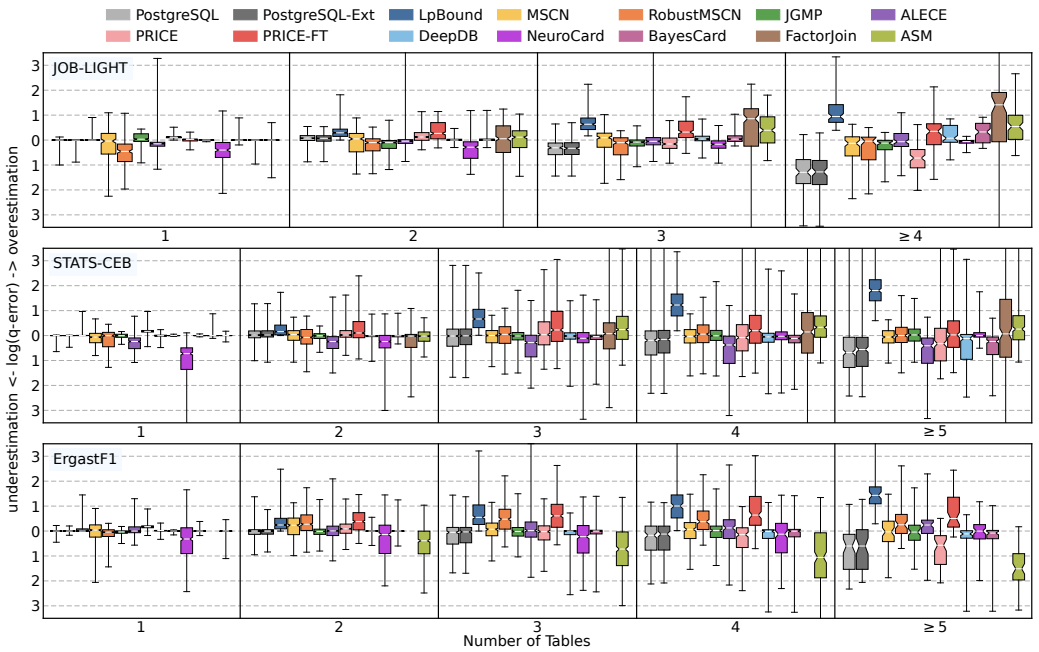


Fig. 2. Q-Error distributions for varying numbers of tables $|\mathcal{T}_Q|$ on different workloads. Box edges represent the 25th and 75th percentiles, whiskers extend to the 1st and 99th percentiles, and the line inside each box indicates the median.

4.1 Estimation Accuracy

Figure 2 shows the distributions of logarithmic Q-Errors for LCE methods and non-learning baselines. We omit the results for FactorJoin on ErgastF1 because its implementation assumes that each table contains at most two join keys, which does not hold in this dataset. PRICE performs zero-shot estimation using the pretrained model, while PRICE-FT is fine-tuned on a specific database using the same training queries as other query-driven methods. In the following, we first summarize several **overall observations (O)** from the experimental results and then analyze them in detail.

O1: Most learned cardinality estimators consistently outperform PostgreSQL for large queries (e.g., $|\mathcal{T}_Q| \geq 4$) across all evaluated workloads, while PostgreSQL remains competitive for small queries. PostgreSQL provides relatively accurate estimates for small queries, but its estimates increasingly deviate as the query size $|\mathcal{T}_Q|$ grows, exhibiting systematic underestimation for larger joins due to the simplified assumptions. PostgreSQL-Ext slightly improves over PostgreSQL by capturing intra-table correlations, but the improvement is marginal because extended statistics cannot capture cross-table correlations in multi-table joins. In contrast, LCE methods can produce substantially more accurate estimates for larger queries by relaxing these assumptions and capturing complex correlations through learned models.

O2: Among query-driven methods, JGMP achieves the lowest estimation error across all evaluated workloads. This superiority over other learned counterparts highlights the effectiveness of its join graph query representation. PRICE in the zero-shot setting attains accuracy comparable to PostgreSQL. Fine-tuning the pretrained model further improves its performance, although it still falls short of database-specific query-driven methods. ALECE exhibits relatively large Q-Errors on STATS-CEB, suggesting that flat vector representations may be insufficient to capture complex join

structures. Additionally, RobustMSCN and PRICE-FT incur large overestimation on ErgastF1 due to their strong sensitivity to the training query quality, as further analyzed in Section 5.3.

O3: Among data-driven methods, DeepDB and BayesCard generally achieve the highest estimation accuracy. This can be attributed to the effectiveness of their correlated grouping strategy, which offers a favorable trade-off between capturing multi-table correlations and preserving information about single-table distributions. While NeuroCard produces highly accurate estimates for large queries, its performance degrades on smaller queries. This behavior stems from its universal grouping strategy: it models the full outer join of all tables, and computing cardinalities for smaller queries requires downscaling the join fanout. FactorJoin and ASM exhibit higher errors, which are associated with their singleton grouping strategy that trades accuracy for efficiency. FactorJoin is further limited by its independence assumption over join keys, while ASM can suffer from sampling failures [40, 65].

4.2 Plan Quality

We evaluate the quality of query plans derived from the estimated cardinalities using P-Error, as shown in Figure 3. To better characterize the gap between subquery-level estimation quality and the quality of the selected plans, we additionally report subquery rank correlation and plan rank

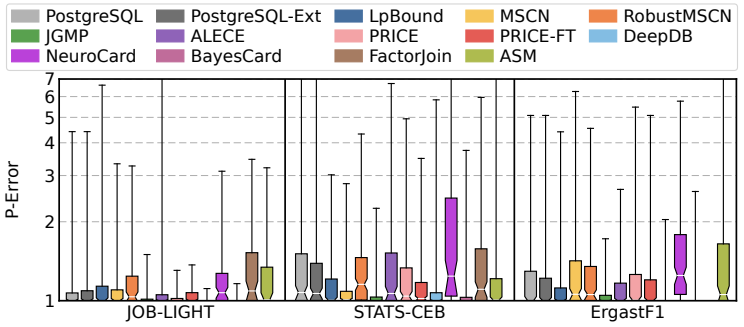


Fig. 3. P-Error distributions on different workloads.

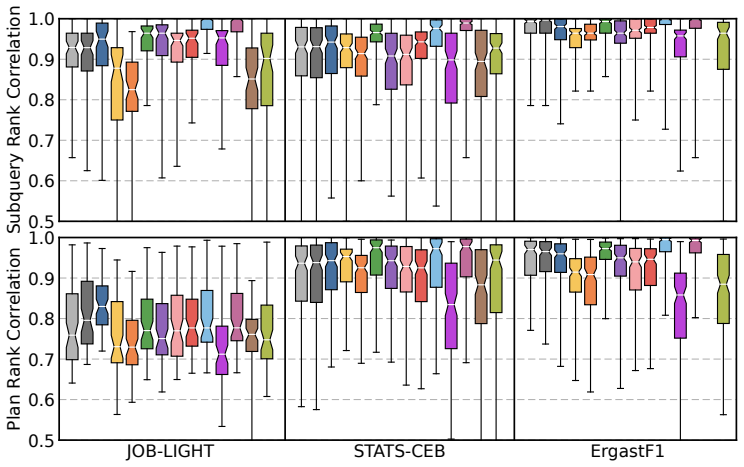


Fig. 4. Subquery rank correlation and plan rank correlation on different workloads.

correlation (Section 3.2) in Figure 4. For queries with more than 10^5 candidate plans, we retain 10^5 plans to compute plan rank correlation.

O4: Most learned estimators achieve P-Errors comparable to or lower than the PostgreSQL baseline, suggesting that LCE methods can improve plan quality. However, this advantage varies across methods and workloads, and certain methods exhibit higher P-Errors, especially on complex workloads with more tables and joins. For example, on ErgastF1, the 99th-percentile P-Errors of MSCN, NeuroCard, and ASM exceed the PostgreSQL baseline.

Among query-driven methods, JGMP achieves the lowest P-Errors across all three workloads and outperforms PostgreSQL. It also achieves high consistency in subquery cardinality estimation and performs strongly in ranking candidate plans, consistent with its geometric learning principles that emphasize invariants in cardinality estimation and encourage more consistent predictions [74]. On JOB-LIGHT and STATS-CEB, the majority of query-driven methods outperform PostgreSQL. In contrast, this improvement is less pronounced on ErgastF1, where only JGMP and ALECE achieve substantially lower P-Errors. This is likely because ErgastF1 contains more tables and joins, posing a greater challenge for query-driven methods to learn diverse join patterns.

For data-driven methods, DeepDB and BayesCard achieve the lowest P-Errors. In contrast, NeuroCard exhibits large P-Errors and performs worse than PostgreSQL across all three workloads, despite being among the most accurate estimators for large queries. This is due to large errors on small subqueries, which adversely impact the selection of high-quality plans. Additionally, FactorJoin and ASM are inferior on STATS-CEB and ErgastF1, respectively, consistent with their estimation accuracy.

4.3 End-to-End Evaluation

We report the end-to-end runtime in Table 4. TrueCard uses ground-truth cardinalities for query optimization and serves as an oracle reference. To provide a broader comparison with other ML-aided query optimization paradigms, we additionally include DACE [46], an LCM approach, and GenJoin [78], an LQO method in the evaluation. We pretrain DACE on 19 datasets released in its official repository that do not overlap with our test datasets. For plan generation, we enumerate all join orders and use DACE to select the best one. GenJoin requires 200 random plans and their execution times for each training query. We randomly select 500 queries from our training set, yielding 10^5 plan–time pairs in total.

O5: Learned estimators can reduce end-to-end runtime compared with PostgreSQL. However, consistently outperforming PostgreSQL across diverse workloads remains difficult. On STATS-CEB, all LCE methods except PRICE significantly reduce PostgreSQL's end-to-end time by 32.3%–57.6%, while PRICE also achieves a marginal improvement of 1.9%. On JOB-LIGHT, all learned estimators except MSCN and PRICE-FT reduce PostgreSQL end-to-end runtime. Similarly, on ErgastF1, all LCE approaches except ASM outperform PostgreSQL in terms of end-to-end time, although the improvements of many methods are marginal. As we further show in Section 5.4, increases in database scale and query complexity pose significant challenges to learned estimators, and the majority of them fail to outperform PostgreSQL. Notably, LpBound consistently reduces PostgreSQL end-to-end runtime across all three workloads and outperforms many LCE methods. In some cases, cardinality estimators even outperform TrueCard in execution time (e.g., JGMP on ErgastF1) due to limitations of the cost model, which may not align with actual execution time [31].

Among query-driven methods, JGMP achieves the best performance, consistently reducing PostgreSQL end-to-end runtime and approaching the TrueCard runtime. RobustMSCN and ALECE also outperform PostgreSQL across all three workloads, while MSCN and PRICE-FT perform slightly worse than PostgreSQL on JOB-LIGHT. PRICE already outperforms PostgreSQL on JOB-LIGHT and STATS-CEB, and further fine-tuning would achieve better performance on STATS-CEB. However,

Table 4. End-to-end runtimes of different methods (seconds). The end-to-end time (E2E) is the sum of plan execution time (Exec) and inference time (Infer). All values are aggregated over all queries in each workload.

Category	Method	JOB-LIGHT			STATS-CEB			ErgastF1		
		E2E	Exec	Infer	E2E	Exec	Infer	E2E	Exec	Infer
Traditional CE	PostgreSQL	1138	1138	–	7068	7068	–	6961	6961	–
	PostgreSQL-Ext	1127	1127	–	7056	7056	–	7000	7000	–
Pessimistic CE	LpBound	996	996	0.099	3012	3011	0.409	6252	6252	0.208
Query-driven LCE	MSCN	1143	1142	1.169	3298	3293	4.742	6622	6618	3.966
	RobustMSCN	992	990	1.361	3319	3313	5.383	6882	6880	1.718
	JGMP	996	995	0.769	3106	3104	1.651	6211	6207	3.802
	ALECE	962	962	0.630	3240	3239	1.065	6824	6822	1.125
	PRICE	1038	1030	8.521	6937	6887	50.479	6951	6934	16.713
	PRICE-FT	1177	1170	6.507	4784	4736	47.499	6952	6937	15.250
Data-driven LCE	DeepDB	974	972	1.749	3100	3081	18.955	6266	6257	8.641
	NeuroCard	1099	1084	15.348	3401	3276	125.13	6775	6537	238.41
	BayesCard	986	980	5.625	3030	3002	28.779	6333	6319	14.314
	FactorJoin	1028	1027	1.328	3766	3756	9.884	–	–	–
	ASM	1090	1088	1.597	2995	2988	7.362	7253	7251	2.105
LCM	DACE	1075	1061	14.009	3799	3451	347.81	7594	7514	80.137
LQO	GenJoin	2002	1999	2.843	10283	10278	4.734	11908	11904	3.687
Reference	TrueCard	931	931	–	2958	2958	–	6403	6403	–

on ErgastF1, both the zero-shot model and the fine-tuned model achieve end-to-end runtimes very close to that of PostgreSQL.

For data-driven methods, DeepDB and BayesCard generally achieve the strongest overall performance and consistently reduce PostgreSQL end-to-end runtime, which aligns with the previously discussed Q-Error and P-Error results. In contrast, ASM does not outperform PostgreSQL on ErgastF1 due to severe underestimation.

For the other ML-aided methods, DACE reduces PostgreSQL's end-to-end runtime on JOB-LIGHT and STATS-CEB but fails to outperform it on ErgastF1. GenJoin, however, does not outperform PostgreSQL on any of the three workloads.

O6: End-to-end workload runtime is not necessarily aligned with Q-Error or P-Error, as these metrics summarize overall estimation accuracy across queries, whereas execution time is often dominated by a small number of long-running queries in the tail. PRICE-FT achieves lower P-Error than PostgreSQL on JOB-LIGHT, but it does not improve end-to-end execution time. This is primarily because it produces a suboptimal plan for a long-running query (Q60) due to significant overestimation, resulting in a runtime of 714.7 seconds compared to 509.0 seconds for the TrueCard baseline. In contrast, although NeuroCard exhibits higher P-Errors than PostgreSQL on STATS-CEB, it still reduces overall runtime, as it selects more efficient plans for the few long-running queries that dominate total runtime.

O7: In terms of estimation efficiency, query-driven methods are fast due to batched GPU inference over subqueries. Data-driven methods vary in efficiency: singleton grouping approaches are the most efficient, followed by correlated grouping methods, while universal grouping methods are the least efficient.

For query-driven methods, feature encoding dominates the estimation overhead. For instance, PRICE incurs higher overhead due to more expensive feature construction, which can be reduced

with further engineering optimizations. For data-driven methods, NeuroCard exhibits the highest overhead due to sampling over a large universal model. In contrast, FactorJoin and ASM achieve the highest efficiency among data-driven methods, as their singleton grouping strategy enables a DP-style estimation process that progressively estimates all subqueries.

5 Practical Considerations

In this section, we examine the practical aspects of LCE methods.

5.1 Model Construction and Storage Cost

Figure 5 summarizes the training time and model size of learned estimators. For query-driven methods, the reported training time excludes the cost of collecting training queries. For PRICE, we report only the fine-tuning time; its pretraining phase takes 21.25 hours on two NVIDIA A40 GPUs.

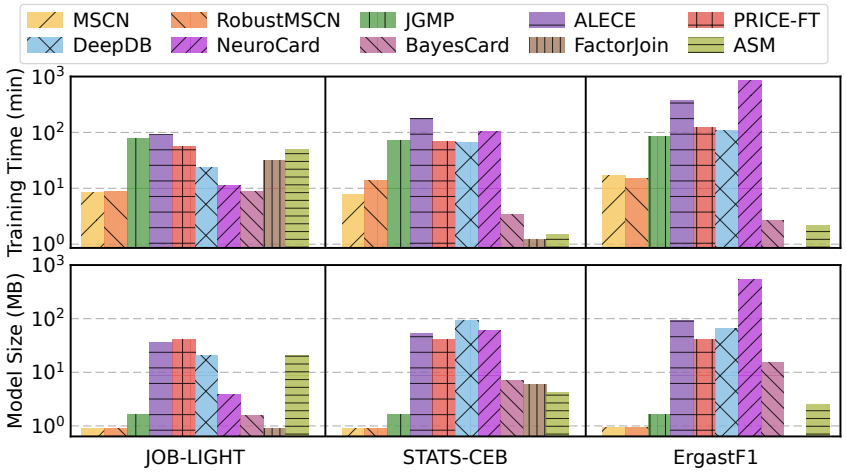


Fig. 5. Training time (min) and model size (MB) of the learned estimators.

O8: Learned estimators introduce non-negligible training time and model storage overhead. For query-driven methods, this overhead is largely determined by the number of model parameters. Among data-driven approaches, BN-based methods generally exhibit lower training cost and smaller model size compared to SPN-based and DAR-based methods. Among query-driven methods, MSCN and RobustMSCN are the most lightweight in both training time and model size due to their small number of model parameters. JGMP has a larger model size and requires more training time than MSCN and RobustMSCN due to the additional sample selection procedure [74] and its more sophisticated model. However, it supports more SQL constructs and yields strong estimation accuracy (Section 4.1) and training data efficiency (Section 5.2). We emphasize that there is a natural trade-off between overhead and capacity and that the overhead results should be interpreted together with the capabilities of each estimator. ALECE and PRICE exhibit the largest model sizes among query-driven methods, along with relatively long training times.

Among data-driven methods, the BN-based methods, BayesCard and FactorJoin, are among the most efficient to train and have small model sizes. FactorJoin is slightly slower on JOB-LIGHT, mainly because bucketing join keys [92] on IMDb is time-consuming due to large attribute domains.

In contrast, DeepDB and NeuroCard incur substantially higher training time and model size on STATS-CEB and ErgastF1. For DeepDB, the overhead stems from the SPN construction process, which involves repeated row clustering and RDC computation between columns. Moreover, the number of SPNs increases with the number of joins, further amplifying the training cost. For NeuroCard, the overhead mainly arises from the large number of attributes and the complexity of the database schema. ASM has small training time and model size on STATS-CEB and ErgastF1, but incurs slightly higher overhead on JOB-LIGHT as larger tables and wider attribute domains lead to larger DAR models.

5.2 Training Data Efficiency of Query-Driven Methods

For query-driven methods, a major practical bottleneck lies in the cost of collecting high-quality training data. We therefore examine how estimator performance scales with the number of training queries. Figure 6 reports the 90th-percentile Q-Error and P-Error results. PostgreSQL-Ext is omitted because its results are very close to those of PostgreSQL. Due to space limitations, the execution time results are provided in Appendix F.1.

O9: Query-driven methods require a large number of training queries to achieve robust performance, which can impose a significant data collection burden in practical deployment. Among individual methods, JGMP achieves lower 90th-percentile Q-Error than PostgreSQL with only about 1000 and 5000 training queries on STATS-CEB and ErgastF1, respectively. On JOB-LIGHT, although JGMP has a higher Q-Error than PostgreSQL with limited training data, it still achieves a lower P-Error. This is consistent with JGMP's geometric learning principle, which emphasizes estimation consistency in addition to relative error [74]. MSCN and RobustMSCN are highly sensitive to training data size, exhibiting poor performance with limited data but improving substantially as more training queries become available. ALECE achieves relatively low Q-Error with small training sets, but its improvement saturates quickly, suggesting that the limited expressiveness of its flat vector feature representation constrains its ultimate accuracy.

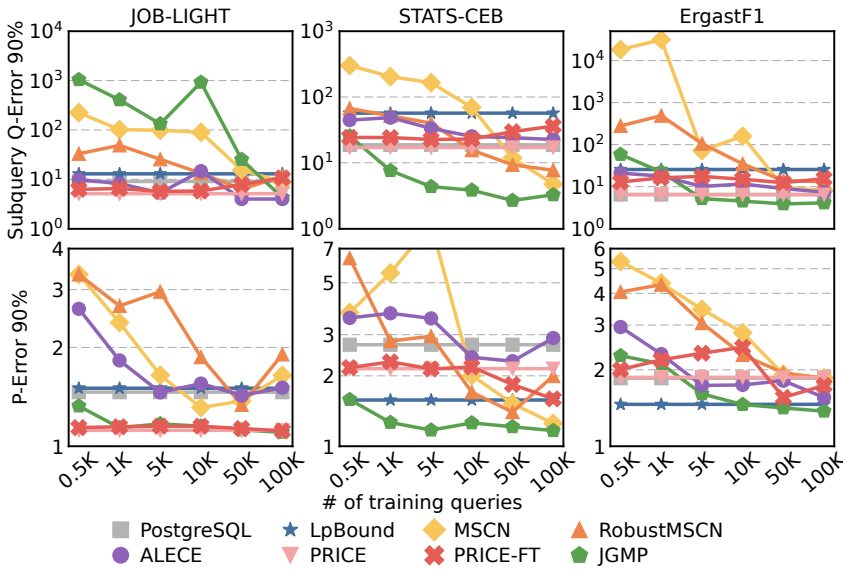


Fig. 6. 90th-percentile Q-Error and P-Error under different training set sizes.

O10: The pretraining paradigm of PRICE requires only a one-time training effort, allows direct predictions on unseen databases, and effectively reduces the dependence on database-specific training workloads. In the zero-shot setting, it achieves slightly lower Q-Error than PostgreSQL. Fine-tuning, however, exhibits unstable effectiveness and depends on the quality of the training queries. This indicates a trade-off between adapting to the target workload and preserving the pretrained knowledge.

5.3 Generalization under Query Workload Drift

Achieving robust generalization under query workload drift [44, 62, 91] is a key challenge for query-driven cardinality estimators. This challenge arises primarily from distributional mismatches between training and inference. In particular, (1) training workloads typically consist of queries with feasible execution times, whereas the optimizer may encounter subqueries with extremely high execution costs in the plan space; and (2) user query patterns may shift abruptly over time. In contrast, data-driven approaches do not rely on a specific training workload and thus do not face this challenge.

Experimental Setup. We evaluate generalization under workload drift using three training configurations with increasing degrees of distribution shift, following a similar setup in prior work [35]. For each configuration, we generate 10^5 training queries, consistent with Section 3.3.

- **Template-based:** We first extract query templates from the test workload, where each template specifies the join structure, filtering columns, and operators, leaving only the filtering literals unspecified. Training queries are then derived from these templates, including all subqueries. This setup approximates a nearly independent and identically distributed (i.i.d.) scenario with minimal distribution shift.
- **Random (Default):** Training queries are generated randomly over the join schema to ensure broad coverage of join patterns, without prior knowledge of the test workload.

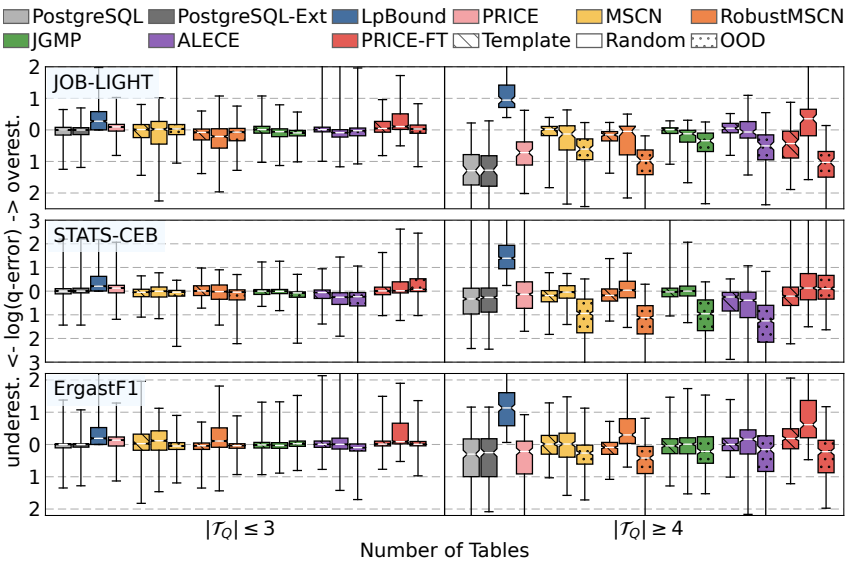


Fig. 7. Q-Error distribution across query sizes and training configurations.

Table 5. Detailed performance of CE methods on JOB-M and JOB-LARGE. E2E time is the sum of cardinality estimation (Est) and plan execution (Exec) time. Encode denotes the feature encoding overhead of query-driven methods and is included in Est. Construction denotes the training time of LCE methods, except for PRICE, for which pretraining time is reported.

Workload	Method	E2E	Exec	Est	Encode	Q-Error					P-Error					Space	Construction
		Time (s)	Time (s)	Time (s)	Time (s)	50%	80%	90%	95%	99%	50%	80%	90%	95%	99%	Cost (MB)	Time (min)
JOB-M	PostgreSQL	2227	2227	–	–	4.69	77.00	467.19	1674.09	$2.17 \cdot 10^4$	1.08	2.47	3.58	4.18	7.66	0.08	0.15
	PostgreSQL-Ext	2256	2256	–	–	3.88	54.16	342.00	1209.00	$2.15 \cdot 10^4$	1.07	2.47	3.46	4.14	6.91	0.08	0.16
	LpBound	2101	2099	1.60	–	13.86	591.85	$3.06 \cdot 10^4$	$9.61 \cdot 10^5$	$4.38 \cdot 10^7$	1.47	6.79	8.79	16.40	26.79	4.54	0.54
	MSCN	2264	2251	12.43	12.16	3.02	10.52	31.26	128.05	1091.15	1.16	2.24	2.85	3.98	6.89	0.93	10.63
	RobustMSCN	2741	2726	15.73	15.17	4.67	25.75	106.28	275.85	1336.26	1.32	2.00	2.97	3.54	6.14	0.94	23.06
	JGMP	2048	2047	1.04	0.29	1.57	12.67	96.13	917.78	$1.67 \cdot 10^4$	1.06	1.86	2.82	3.19	6.47	1.63	72.86
	ALECE	7842	7841	1.63	0.24	5.97	38.87	153.13	902.12	$2.14 \cdot 10^4$	1.50	3.09	4.92	7.55	28.70	51.25	68.49
	PRICE	3468	3397	71.15	67.65	3.16	25.17	70.01	176.50	849.78	1.38	3.91	5.47	5.88	10.52	41.12	1274.93
	PRICE-FT	2211	2145	65.84	62.56	4.15	22.33	63.18	188.83	1480.81	1.05	3.21	6.07	12.72	36.94	41.13	207.52
	DeepDB	2064	2057	6.71	–	1.48	11.30	1582.00	$1.15 \cdot 10^4$	$1.05 \cdot 10^5$	1.02	2.45	6.34	18.60	153.37	139.41	48.87
	NeuroCard	3735	3333	401.32	–	121.80	425.15	1155.77	2707.67	$3.98 \cdot 10^4$	2.26	5.17	7.61	20.82	5201.47	27.30	49.29
	BayesCard	2066	2025	40.96	–	1.36	4.51	18.95	47.26	4072.62	1.00	1.09	1.90	3.05	5.43	9.91	31.67
	ASM	2219	2210	9.14	–	4.82	14.26	44.38	108.45	833.17	1.06	1.47	1.77	2.30	4.36	44.18	34.26
	TrueCard	2109	2109	–	–	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	–	-
JOB-LARGE	PostgreSQL	6376	6376	–	–	56.61	1833.50	$1.44 \cdot 10^4$	$6.11 \cdot 10^4$	$7.33 \cdot 10^5$	1.10	2.16	3.65	8.60	246.79	0.13	0.19
	PostgreSQL-Ext	4990	4990	–	–	49.49	1701.88	$1.38 \cdot 10^4$	$6.11 \cdot 10^4$	$7.06 \cdot 10^5$	1.15	2.21	4.05	15.77	222.49	0.23	0.51
	LpBound	5888	5755	132.75	–	1508.41	$9.48 \cdot 10^4$	$8.33 \cdot 10^5$	$7.31 \cdot 10^6$	$2.42 \cdot 10^8$	1.28	3.69	10.03	19.64	39.91	38.04	2.78
	MSCN	31218	29876	1342	1333	105.54	$1.76 \cdot 10^4$	$6.86 \cdot 10^5$	$6.41 \cdot 10^6$	$2.22 \cdot 10^8$	1.63	6.54	24.74	96.27	535.36	0.95	16.00
	RobustMSCN	19047	9279	9769	9760	27.78	577.29	2822.42	8842.08	$4.92 \cdot 10^4$	1.73	4.61	9.11	13.51	93.40	0.96	36.97
	JGMP	4759	4707	51.48	46.50	69.63	2272.55	$1.50 \cdot 10^4$	$7.17 \cdot 10^4$	$6.69 \cdot 10^5$	1.18	2.34	4.09	6.75	29.13	1.64	62.52
	ALECE	15223	14991	231.78	14.92	15.24	258.39	1870.91	$1.24 \cdot 10^4$	$1.24 \cdot 10^5$	1.33	5.86	14.15	33.71	294.05	105.88	268.21
	PRICE	11220	7342	3878	3789	9.55	86.79	357.05	1243.87	$1.68 \cdot 10^4$	1.30	3.59	9.54	20.96	79.37	41.12	1274.93
	PRICE-FT	11674	7787	3887	3796	39.06	486.64	1956.37	7013.53	$1.33 \cdot 10^5$	1.31	2.29	4.58	14.93	55.21	41.13	162.20
	DeepDB	17529	17053	475.91	–	162.17	$3.06 \cdot 10^4$	$6.85 \cdot 10^5$	$5.21 \cdot 10^6$	$1.07 \cdot 10^8$	1.03	2.29	3.84	157.90	1102.23	2948	214.17
	NeuroCard	276564	17994	258570	–	252.93	3254.09	$2.12 \cdot 10^4$	$7.33 \cdot 10^4$	$5.87 \cdot 10^5$	4.61	12.76	27.17	69.05	114.21	134.30	298.71
	BayesCard	50178	28067	22111	–	86.78	$5.75 \cdot 10^4$	$8.43 \cdot 10^5$	$8.85 \cdot 10^6$	$2.77 \cdot 10^8$	1.13	2.45	8.99	257.74	1139.40	47.23	220.20
	ASM	14806	14717	88.44	–	27.02	382.00	3234.00	$2.76 \cdot 10^5$	$3.89 \cdot 10^7$	1.25	2.35	4.13	5.49	83.48	75.55	45.29
	TrueCard	3806	3806	–	–	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	–	-

- **Out-of-Distribution (OOD):** To simulate workload drift, the training set is restricted to queries with 0–2 joins (i.e., $1 \leq |\mathcal{T}_Q| \leq 3$), while the test workload contains queries with more joins.

Experimental Results. Figure 7 reports the Q-Error distributions under the three training configurations. Due to space limits, P-Error and execution time results are provided in Appendix F.2.

O11: Query-driven methods are highly sensitive to training data quality, highlighting the need for high-quality training queries to ensure robust performance. Across all training configurations, each estimator exhibits comparable performance on small queries ($|\mathcal{T}_Q| \leq 3$), with Template and OOD in general outperforming Random. For larger queries ($|\mathcal{T}_Q| \geq 4$), Template generally achieves the highest accuracy, followed by Random, while OOD exhibits the most pronounced degradation. Under the OOD configuration, accuracy deteriorates rapidly as query complexity increases, reflecting the absence of representative complex queries in the training set.

5.4 Scalability

As databases and workloads grow, data correlations become harder to model and queries involve larger joins with up to tens of thousands of subqueries. This poses two challenges: modeling high-dimensional correlated data and efficiently estimating massive numbers of subqueries.

We evaluate scalability on JOB-M and JOB-LARGE, two workloads that stress both schema and query complexity. Compared to JOB-LIGHT, they increase the number of tables from 6 to 16 and 21, respectively, while queries involve up to 11 and 19 joined tables. Results are summarized in Table 5. For JOB-LARGE, queries exceeding a 5-hour timeout are recorded as 5 hours. FactorJoin is excluded because its implementation assumes at most two join keys per table, which does not hold on these workloads.

O12: Outperforming PostgreSQL on large-scale databases and highly complex queries remains difficult, as many methods fail to simultaneously maintain estimation accuracy and efficiency at scale, highlighting a persistent gap between current learned estimators and real-world scalability requirements.

Results on JOB-M. All query-driven methods achieve lower Q-Error than PostgreSQL. However, JGMP is the only method that significantly reduces end-to-end time. PRICE-FT reduces plan execution time but achieves only marginal improvement in end-to-end time because of its high estimation overhead, while the remaining query-driven methods fail to outperform PostgreSQL. This observation again highlights the gap between estimation accuracy and optimization effectiveness: good Q-Error alone does not guarantee high-quality plans [21, 61]. For data-driven methods, DeepDB, BayesCard, and ASM outperform PostgreSQL in end-to-end runtime. In contrast, NeuroCard performs worse than PostgreSQL due to large estimation errors on small queries.

Feature encoding constitutes a major portion of the estimation time for query-driven methods, primarily due to expensive feature construction, such as bitmap generation in RobustMSCN.

Results on JOB-LARGE. JOB-LARGE represents an extreme scalability challenge, with up to 19 joined tables and 50,000 subqueries per query. Surprisingly, PostgreSQL-Ext reduces PostgreSQL's end-to-end runtime by 21.7% on this workload, mainly due to the successful optimization of two long-running queries. Among query-driven methods, only JGMP reduces PostgreSQL's end-to-end runtime, by 25.4%. All data-driven methods exhibit higher Q-Errors than PostgreSQL and incur execution times approximately 2.3–4.4× longer. These results suggest that the large-scale, highly correlated data in this workload are difficult to model, posing a significant challenge for data-driven approaches.

5.5 Adaptability to Dynamic Updates

Dynamic workloads introduce continuous distribution shifts, making it challenging for learned estimators to remain accurate over time. An effective estimator in such settings should therefore provide both *efficient updates* and *minimal accuracy degradation*, ideally achieving performance comparable to retraining from scratch.

Experimental Setup. We evaluate dynamic updates on the IMDb and STATS datasets, both containing temporal attributes that allow the construction of database snapshots. Given a timestamp t, each table is filtered by its temporal attribute, after which PK–FK constraints are propagated to construct the initial database $D_{\leq t}$. Details are provided in Appendix E. We define the initial data ratio as $\rho = \sum_{T \in \mathcal{T}_{D_{\leq t}}} |T| / \sum_{T \in \mathcal{T}_D} |T|$, which represents the fraction of tuples in the initial database. We vary ρ over $\{20\%, 40\%, 60\%, 80\%\}$ to evaluate robustness under different update magnitudes. For each ρ, we select a timestamp t_ρ such that the resulting ratio approximates ρ, and denote the corresponding initial database as D^ρ (i.e., $D_{\leq t_\rho}$) and the remaining data as $\Delta D^\rho = D - D^\rho$.

For query-driven methods, training requires query workloads over multiple snapshots. Specifically, for each D^ρ, we construct ten preceding snapshots $D^{\rho \cdot i/10}$, where $i = 1, \ldots, 10$. On each snapshot, we generate 10^4 training queries, resulting in a total of 10^5 queries, consistent with the settings used in Section 3.3.

For data-driven methods, models are initially trained on D^ρ and updated when ΔD^ρ arrives. For correlated grouping methods, this requires determining how the full outer join of each group changes. Consider a group $G = \{S, T\}$ at timestamp t: the model is trained on $S_{\leq t} \bowtie T_{\leq t}$, and updating it requires the change set $\Delta(S \bowtie T)$.

Among existing approaches, only DeepDB implements this process. It first materializes the full outer join $S \bowtie T$ over the post-update database. It then applies temporal filtering predicates on the temporal columns of the join result: tuples satisfying the condition $\leq t$ are treated as $S_{\leq t} \bowtie T_{\leq t}$, while the remaining tuples are regarded as $\Delta(S \bowtie T)$. However, this implementation relies on future update information, may yield incorrect initial joins and change sets, and is restricted to specific temporal update assumptions rather than supporting general update scenarios, as detailed

in Appendix B. Given these limitations, we introduce two practical alternatives. Together with the original DeepDB strategy, we evaluate the following three update mechanisms.

- **Incremental Update (INC).** Given updates ΔS and ΔT, we explicitly compute the changes $\Delta(S \bowtie T)$ from $(S_{\leq t}, T_{\leq t}, \Delta S, \Delta T)$ and apply incremental updates to the model.
- **Parameter Refit (REF).** Instead of explicitly materializing $\Delta(S \bowtie T)$, this strategy recomputes $S \bowtie T$ on the updated database and refits only the model parameters while keeping the learned SPN or BN structure fixed.
- **Temporal Filtering (TF).** The original DeepDB strategy based on temporal filtering, which is included solely for comparison.

Experimental Results. Figure 8 reports the update time results.

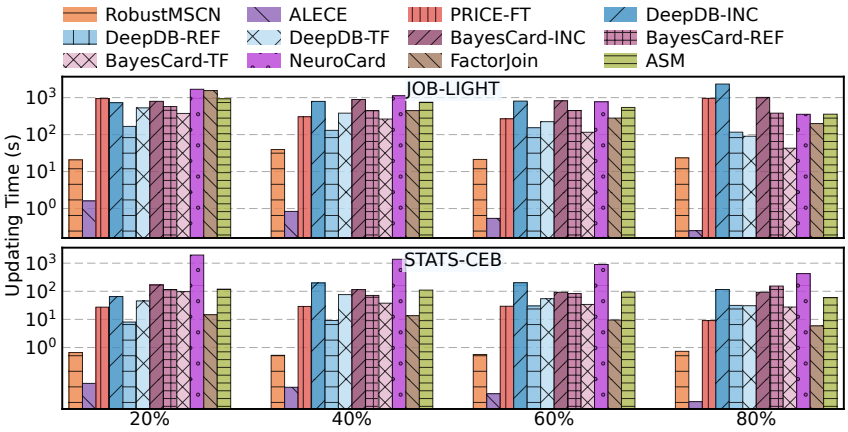


Fig. 8. Update time of the learned estimators under different initial data ratios.

O13: Most learned estimators support efficient updates, often several orders of magnitude faster than retraining from scratch. Update times on JOB-LIGHT are generally higher than on STATS-CEB, mainly due to the larger table sizes in IMDb.

Among query-driven methods, ALECE and RobustMSCN exhibit the lowest update overhead because they only refresh lightweight statistics such as histograms or database samples. PRICE is comparatively slower due to maintaining richer statistics such as join scaling factors. For data-driven methods, correlated grouping and universal grouping methods generally incur higher update costs than singleton grouping methods. Among correlated grouping methods, Parameter Refit can be more efficient than Incremental Update, suggesting that the change set of the full outer join can be very large, potentially even larger than the updated full outer join.

Figure 9 shows the Q-Error distribution of updated models. For reference, we additionally report results for stale models without updates and freshly retrained models. The P-Error and execution time results are provided in Appendix F.3 due to space limits.

O14: Most learned estimators experience noticeable accuracy degradation after updates compared to retraining from scratch. Among query-driven methods, only PRICE-FT shows negligible degradation. Among data-driven methods, DeepDB and BayesCard achieve the most stable performance, with accuracy close to freshly retrained models.

Query-driven methods aim to learn a mapping from queries Q and database statistics S_D to cardinalities, implicitly assuming that the training samples (Q, S_D) are i.i.d. In practice, however, S_D may exhibit correlations and evolve over time, violating this assumption under dynamic updates,

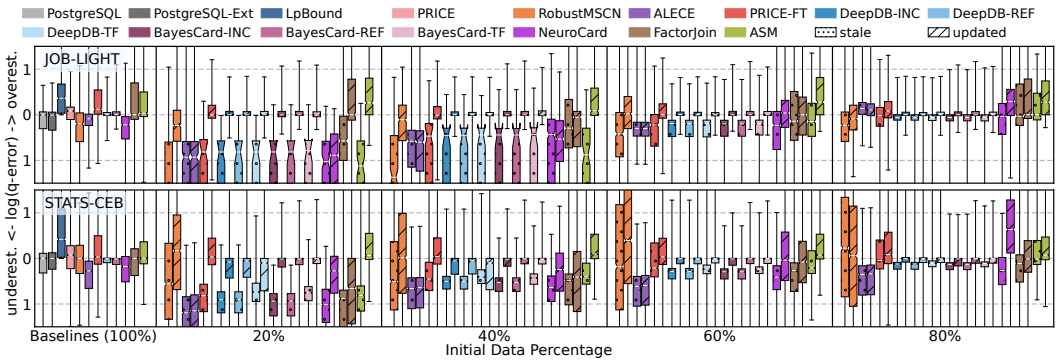


Fig. 9. Q-Error distributions under varying initial data ratios for the stale model (dotted hatching), the updated model (diagonal hatching), and the retrained fresh model (the leftmost "Baselines" without hatching).

leading to significant degradation for RobustMSCN and ALECE. In contrast, PRICE-FT maintains stable performance due to transferable pretrained knowledge.

For data-driven methods, all variants of DeepDB and BayesCard achieve Q-Error distributions close to those of freshly retrained models. DAR-based methods, including NeuroCard and ASM, show moderate degradation because fine-tuning intentionally trades accuracy for efficiency by using only a limited number of training tuples. Updated FactorJoin models also underperform their fully retrained counterparts, as join-key buckets constructed from partial data are less representative than those built from the complete data.

5.6 Integration with Traditional Methods

We investigate whether learned and traditional cardinality estimators can be effectively combined to improve estimation performance. We present a case study for query-driven methods here and provide another for data-driven methods in Appendix F.4.

We introduce MSCN-Hybrid, which augments MSCN with an uncertainty score and falls back to PostgreSQL when the uncertainty exceeds a threshold. Following prior work [27, 29, 100], we quantify both *epistemic* and *aleatoric uncertainty*. For epistemic uncertainty U_E, we replace MSCN's original prediction MLP with k independently initialized heads and use the mean and variance of their predictions as the final estimate $\hat{y}$ and U_E, respectively. For aleatoric uncertainty U_A, we add an additional MLP head to predict U_A and train it with the Gaussian negative log-likelihood [63] in Equation (2), where q_i and y_i denote the encoded query embedding and ground-truth label.

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{\log U_A(q_i)}{2} + \frac{(y_i - \hat{y}_i)^2}{2U_A(q_i)} + \frac{\log 2\pi}{2}\right). \tag{2}$$

Finally, we use $U = U_E + U_A$ as the total uncertainty and fall back to PostgreSQL when $U > U_0$. Figure 10 reports the 90th-percentile Q-Error with $U_0 = 0.005$. MSCN-Hybrid consistently improves over MSCN across different training set sizes, demonstrating the potential of an uncertainty-aware fallback strategy.

6 Related Work

Learned Cardinality Estimation (LCE). We first review other LCE methods for SPJ queries. UAE [90] employs a deep autoregressive model and represents a hybrid approach that learns from both data and queries. Fauce [48] and NNGP [106] are uncertainty-aware query-driven estimators based

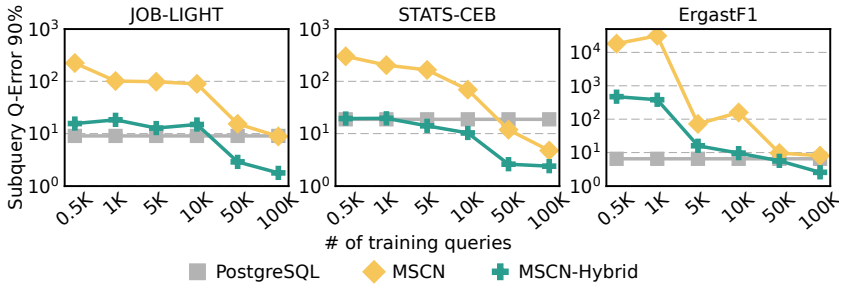


Fig. 10. 90th-percentile Q-Error of PostgreSQL, MSCN and MSCN-Hybrid under different training set sizes.

on deep ensembles and neural network Gaussian processes, respectively. FLAT [113] extends the SPN model used in DeepDB [26] to a Factorize-Sum-Split-Product Network (FSPN). In addition, a large body of work focuses exclusively on single-table cardinality estimation [13, 66, 88, 97, 105, 114]. Recently, several studies have further expanded the scope of supported query types, such as string LIKE predicates [2, 33, 103] and set-valued attributes [76].

Learned Cost Models (LCM). These methods [4, 24, 25, 46, 54, 79, 107] apply machine learning models to directly predict the execution cost of a query plan. They can be categorized into two types: DB-specific and DB-agnostic [24]. DB-specific methods [4, 54, 79, 107] are trained on pairs of query plans and their execution times from a single database and can only make predictions on that database. DB-agnostic methods [25, 46] are trained on multiple databases and can generalize to previously unseen databases. A recent experimental study [24] shows that although LCMs achieve high prediction accuracy, they are still inferior to the traditional PostgreSQL cost model in terms of plan ranking and selection.

Learned Query Optimization (LQO). These methods generate execution plans in a direct end-to-end fashion [6, 7, 53, 95, 101], or guide plan generation in an indirect manner [43, 52, 78, 100, 110], such as hint generation [52, 78] or candidate plan selection [43, 110]. LQO techniques have also been applied in other query paradigms and execution environments, including graph queries [87, 94], parametric queries [12, 34, 45, 55, 84], and Spark SQL workloads [23, 99]. However, a recent experimental study [35] shows that existing LQO approaches do not consistently outperform PostgreSQL. Recently, large language models (LLMs) have also been explored for query optimization [1, 41, 47, 49, 81, 98], mainly focusing on hint generation [47, 49] and candidate plan selection [98].

7 Conclusions and Future Directions

This paper presents a systematic experimental study of learned cardinality estimation (LCE) for selection–projection–join (SPJ) queries. We conduct a comprehensive, multi-dimensional evaluation of ten representative LCE approaches across three real-world datasets, examining both overall effectiveness for query optimization and practical deployment considerations, and derive a set of empirical observations that reveal the strengths and limitations of current techniques. Finally, we summarize the key takeaways of our study and highlight several directions for future research.

7.1 Key Takeaways

- *Neither query-driven nor data-driven methods clearly dominate.* Query-driven methods can exploit workload information to achieve high accuracy while remaining lightweight and efficient at inference, but typically require sufficient high-quality training queries. Data-driven methods are attractive when training queries are unavailable, yet may incur higher modeling overhead and face accuracy challenges on large, highly correlated, or skewed datasets. In dynamic settings,

however, data-driven methods, particularly correlated grouping approaches, can adapt directly to data changes without relabeling training queries and often exhibit less degradation after updates.
- *Multi-dimensional evaluation is essential.* Estimation accuracy alone provides an incomplete view of LCE performance. Plan-ranking ability, selected plan quality, end-to-end runtime, deployment overhead, robustness across heterogeneous workloads and datasets, and adaptation to data updates can all substantially affect practical effectiveness. Our results further show that newer methods are not necessarily superior to earlier ones across these dimensions, highlighting the importance of holistic evaluation rather than relying on a single metric.
- *Consistently outperforming traditional estimators remains challenging.* Despite substantial progress in learned cardinality estimation, no learned estimator consistently dominates traditional approaches across workloads and settings. The greater expressive power of LCE methods can provide significant benefits under favorable conditions, but has not yet translated into consistent superiority across evaluation dimensions.

7.2 Future Directions

- *Improving estimation consistency.* Most LCE methods optimize pointwise estimation accuracy, while consistency among subquery estimates can also affect plan ranking and selection. Future LCE methods could therefore explicitly encourage consistent subquery estimates to improve plan-ranking performance.
- *Avoiding silent failure.* Most LCE methods output cardinality estimates without indicating when these estimates may be unreliable. Estimators that jointly provide cardinalities and uncertainty could enable fallback to traditional methods, support risk-aware query optimization, facilitate failure diagnosis, and guide the collection of informative training queries.
- *Exploiting subquery overlap.* Estimating all subqueries independently can be expensive, especially for large join queries. Singleton grouping methods already exploit shared computation through DP-style progressive estimation. Similar ideas could benefit other methods, e.g., by reusing feature representations across overlapping subqueries in query-driven methods or caching shared factors in correlated grouping methods.

Acknowledgments

We sincerely thank the anonymous reviewers for their valuable and constructive feedback. This work was supported by the National Key Research and Development Program of China under grant 2024YFF0907603 and the National Natural Science Foundation of China under grant 62532001. Lei Zou is the corresponding author of this work.

References

[1] Peter Akioyamen, Zixuan Yi, and Ryan Marcus. 2024. The unreasonable effectiveness of LLMs for query optimization. *arXiv preprint arXiv:2411.02862* (2024).

[2] Mehmet Aytimur, Silvan Reiner, Leonard Wörteler, Theodoros Chondrogiannis, and Michael Grossniklaus. 2024. Lplm: A neural language model for cardinality estimation of like-queries. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–25.

[3] Ashok K Chandra and Philip M Merlin. 1977. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of the ninth annual ACM symposium on Theory of computing*. 77–90.

[4] Baoming Chang, Amin Kamali, and Verena Kantere. 2026. Reqo: A Comprehensive Learning-Based Cost Model for Robust and Explainable Query Optimization. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–27.

[5] Surajit Chaudhuri. 1998. An overview of query optimization in relational systems. In *Proceedings of the seventeenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 34–43.

[6] Tianyi Chen, Jun Gao, Hedui Chen, and Yaofeng Tu. 2023. Loger: A learned optimizer towards generating efficient and robust query execution plans. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1777–1789.

[7] Tianyi Chen, Jun Gao, Yaofeng Tu, Yang Lin, Mo Xu, and Yinjun Han. 2026. Divo: Learning a Stable and Effective Query Optimizer with a Diverse Workload. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–24.

[8] CTU Relational Dataset Repository. 2025. ErgastF1 Dataset. https://relational.fel.cvut.cz/dataset/ErgastF1. Accessed: 2025.

[9] CTU Relational Dataset Repository. 2025. Stats Dataset. https://relational.fel.cvut.cz/dataset/Stats. Accessed: 2025.

[10] Amol Deshpande, Minos Garofalakis, and Rajeev Rastogi. 2001. Independence is good: Dependency-based histogram synopses for high-dimensional data. *ACM SIGMOD Record* 30, 2 (2001), 199–210.

[11] Bolin Ding, Rong Zhu, and Jingren Zhou. 2024. Learned Query Optimizers. *Foundations and Trends® in Databases* 13, 4 (2024), 250–310.

[12] Lyric Doshi, Vincent Zhuang, Gaurav Jain, Ryan Marcus, Haoyu Huang, Deniz Altinbüken, Eugene Brevdo, and Campbell Fraser. 2023. Kepler: Robust learning for parametric query optimization. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–25.

[13] Anshuman Dutt, Chi Wang, Azade Nazi, Srikanth Kandula, Vivek Narasayya, and Surajit Chaudhuri. 2019. Selectivity estimation for range predicates using lightweight models. *Proceedings of the VLDB Endowment* 12, 9 (2019), 1044–1057.

[14] Ahmed K Farahat, Ali Ghodsi, and Mohamed S Kamel. 2013. Efficient greedy feature selection for unsupervised learning. *Knowledge and information systems* 35, 2 (2013), 285–310.

[15] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. 2015. MADE: Masked autoencoder for distribution estimation. In *International conference on machine learning*. PMLR, 881–889.

[16] Lise Getoor, Benjamin Taskar, and Daphne Koller. 2001. Selectivity estimation using probabilistic models. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 461–472.

[17] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. Pmlr, 1263–1272.

[18] Goetz Graefe. 1995. The cascades framework for query optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29.

[19] Goetz Graefe and William J McKenna. 1993. The volcano optimizer generator: Extensibility and efficient search. In *Proceedings of IEEE 9th international conference on data engineering*. IEEE, 209–218.

[20] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[21] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. *Proceedings of the VLDB Endowment* 15, 4 (2021), 752–765. doi:10.14778/3503585.3503586

[22] Trevor Hastie, Jerome H. Friedman, and Robert Tibshirani. 2001. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer. doi:10.1007/978-0-387-21606-5

[23] Jiahao He, Yutao Cui, Cuiping Li, Jikang Jiang, Yuheng Hou, and Hong Chen. 2025. AQORA: A Learned Adaptive Query Optimizer for Spark SQL. arXiv:2510.10580v1 [cs.DB] https://arxiv.org/abs/2510.10580v1

[24] Roman Heinrich, Manisha Luthra, Johannes Wehrstein, Harald Kornmayer, and Carsten Binnig. 2025. How good are learned cost models, really? Insights from query optimization tasks. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.

[25] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-shot cost models for out-of-the-box learned cost prediction. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2361–2374.

[26] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2020. DeepDB: learn from data, not from queries! *Proceedings of the VLDB Endowment* 13, 7 (2020), 992–1005.

[27] Eyke Hüllermeier and Willem Waegeman. 2021. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine learning* 110, 3 (2021), 457–506.

[28] Yannis E Ioannidis and Stavros Christodoulakis. 1993. Optimal histograms for limiting worst-case error propagation in the size of join results. *ACM Transactions on Database Systems (TODS)* 18, 4 (1993), 709–748.

[29] Alex Kendall and Yarin Gal. 2017. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems* 30 (2017).

[30] Kyoungmin Kim, Jisung Jung, In Seo, Wook-Shin Han, Kangwoo Choi, and Jaehyok Chong. 2022. Learned cardinality estimation: An in-depth study. In *Proceedings of the 2022 international conference on management of data*. 1214–1227.

[31] Kyoungmin Kim, Sangoh Lee, Injung Kim, and Wook-Shin Han. 2024. ASM: Harmonizing autoregressive model, sampling, and multi-dimensional statistics merging for cardinality estimation. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.

[32] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*.

[33] Suyong Kwon, Kyuseok Shim, and Woohwan Jung. 2025. Cardinality Estimation of LIKE Predicate Queries using Deep Learning. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.

[34] Hai Lan, Yang Yu, Zhifeng Bao, Zi Huang, and Yuwei Peng. 2026. Practical Parameterized Query Optimization via Efficient Plan Reuse and List-wise Ranking. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–26.

[35] Claude Lehmann, Pavel Sulimov, and Kurt Stockinger. 2024. Is Your Learned Query Optimizer Behaving As You Expect? A Machine Learning Perspective. *Proceedings of the VLDB Endowment* 17, 7 (2024), 1565–1577.

[36] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.

[37] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2025. Still Asking: How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 18, 12 (2025), 5531–5536.

[38] Viktor Leis, Bernhard Radke, Andrey Gubichev, Alfons Kemper, and Thomas Neumann. 2017. Cardinality Estimation Done Right: Index-Based Join Sampling.. In *CIDR*.

[39] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query optimization through the looking glass, and what we found running the join order benchmark. *The VLDB Journal* 27, 5 (2018), 643–668.

[40] Feifei Li, Bin Wu, Ke Yi, and Zhuoyue Zhao. 2016. Wander join: Online aggregation via random walks. In *Proceedings of the 2016 International Conference on Management of Data*. 615–629.

[41] Guoliang Li, Xuanhe Zhou, and Xinyang Zhao. 2024. LLM for data management. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4213–4216.

[42] Pengfei Li, Wenqing Wei, Rong Zhu, Bolin Ding, Jingren Zhou, and Hua Lu. 2023. ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads. *Proceedings of the VLDB Endowment* 17, 2 (2023), 197–210.

[43] Runzhong Li, Qilong Li, Haotian Liu, Rui Mao, Qing Li, and Bo Tang. 2025. Athena: An Effective Learning-based Framework for Query Optimizer Performance Improvement. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.

[44] Rui Li, Kangfei Zhao, Jeffrey Xu Yu, and Guoren Wang. 2024. CardOOD: Robust Query-driven Cardinality Estimation under Out-of-Distribution. *arXiv preprint arXiv:2412.05864* (2024).

[45] Sijia Li, Peng Cai, Zhifan Zhang, Huiqi Hu, Rong Zhang, Xuan Zhou, Quanqing Xu, and Chuanhui Yang. 2025. APQO: An Adaptive Framework for Parametric Query Optimization. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[46] Zibo Liang, Xu Chen, Yuyang Xia, Runfan Ye, Haitian Chen, Jiandong Xie, and Kai Zheng. 2024. DACE: A database-agnostic cost estimator. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4925–4937.

[47] Hanwen Liu, Qihan Zhang, Ryan Marcus, and Ibrahim Sabek. 2025. SEFRQO: A Self-Evolving Fine-Tuned RAG-Based Query Optimizer. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–27.

[48] Jie Liu, Wenqian Dong, Qingqing Zhou, and Dong Li. 2021. Fauce: fast and accurate deep ensembles with uncertainty for cardinality estimation. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1950–1963.

[49] Suchen Liu, Jun Gao, Yinjun Han, and Yang Lin. 2025. LLM4Hint: Leveraging Large Language Models for Hint Recommendation in Offline Query Optimization. *arXiv preprint arXiv:2507.03384* (2025).

[50] Pedro Lopes, Craig Guyer, and Milener Gene. 2019. SQL Docs: Cardinality Estimation (SQL Server). https://docs.microsoft.com/en-us/sql/relational-databases/performance/cardinality-estimation-sql-server?view=sql-server-ver15 Accessed: 2025.

[51] David Lopez-Paz, Philipp Hennig, and Bernhard Schölkopf. 2013. The randomized dependence coefficient. *Advances in neural information processing systems* 26 (2013).

[52] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making learned query optimization practical. In *Proceedings of the 2021 International Conference on Management of Data*. 1275–1288.

[53] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1705–1718.

[54] Ryan Marcus and Olga Papaemmanouil. 2019. Plan-structured deep neural network models for query performance prediction. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1733–1746.

[55] Songsong Mo, Yue Zhao, Zhifeng Bao, Quanqing Xu, Chuanhui Yang, and Gao Cong. 2024. RankPQO: Learning-to-Rank for Parametric Query Optimization. *Proceedings of the VLDB Endowment* 18, 3 (2024), 863–875.

[56] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. 2009. Preventing bad plans by bounding the impact of cardinality estimation errors. *Proceedings of the VLDB Endowment* 2, 1 (2009), 982–993.

[57] Alejandro Molina, Sriraam Natarajan, and Kristian Kersting. 2017. Poisson sum-product networks: A deep architecture for tractable multivariate poisson distributions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 31.

[58] M Muralikrishna and David J DeWitt. 1988. Equi-depth multidimensional histograms. In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*. 28–36.

[59] MySQL Documentation. 2024. Configuring Non-Persistent Optimizer Statistics Parameters. MySQL 8.4 Reference Manual, Section 17.8.10.2. https://dev.mysql.com/doc/refman/8.4/en/innodb-statistics-estimation.html Accessed: 2025.
[60] Charlie Nash and Conor Durkan. 2019. Autoregressive energy machines. In *International Conference on Machine Learning*. PMLR, 1735–1744.
[61] Parimarjan Negi, Ryan Marcus, Andreas Kipf, Hongzi Mao, Nesime Tatbul, Tim Kraska, and Mohammad Alizadeh. 2021. Flow-loss: learning cardinality estimates that matter. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2019–2032.
[62] Parimarjan Negi, Ziniu Wu, Andreas Kipf, Nesime Tatbul, Ryan Marcus, Sam Madden, Tim Kraska, and Mohammad Alizadeh. 2023. Robust query driven cardinality estimation under changing workloads. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1520–1533.
[63] David A Nix and Andreas S Weigend. 1994. Estimating the mean and variance of the target probability distribution. In *Proceedings of 1994 ieee international conference on neural networks (ICNN'94)*, Vol. 1. IEEE, 55–60.
[64] Jennifer Ortiz, Magdalena Balazinska, Johannes Gehrke, and S Sathiya Keerthi. 2019. An empirical analysis of deep learning for cardinality estimation. *arXiv preprint arXiv:1905.06425* (2019).
[65] Yeonsu Park, Seongyun Ko, Sourav S Bhowmick, Kyoungmin Kim, Kijae Hong, and Wook-Shin Han. 2020. G-CARE: A framework for performance benchmarking of cardinality estimation techniques for subgraph matching. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1099–1114.
[66] Yongjoo Park, Shucheng Zhong, and Barzan Mozafari. 2020. Quicksel: Quick selectivity learning with mixture models. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1017–1033.
[67] Judea Pearl. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.
[68] Hoifung Poon and Pedro Domingos. 2011. Sum-product networks: A new deep architecture. In *2011 IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*. IEEE, 689–690.
[69] Viswanath Poosala and Yannis E. Ioannidis. 1997. Selectivity estimation without the attribute value independence assumption. In *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases, August 25-29, 1997, Athens, Greece*. 486–495.
[70] PostgreSQL Global Development Group. 2025. CREATE STATISTICS. PostgreSQL 18 Documentation, SQL Commands. https://www.postgresql.org/docs/current/sql-createstatistics.html Accessed: 2026.
[71] PostgreSQL Global Development Group. 2025. pg_prewarm — preload relation data into buffer caches. PostgreSQL 18 Documentation, Chapter F.30. https://www.postgresql.org/docs/current/pgprewarm.html Accessed: 2026.
[72] PostgreSQL Global Development Group. 2025. Row Estimation Examples. PostgreSQL 18 Documentation, Chapter 69.1. https://www.postgresql.org/docs/current/row-estimation-examples.html Accessed: 2026.
[73] Nazanin Rashedi and Guido Moerkotte. 2025. The Accuracy of Cardinality Estimators: Unraveling the Evaluation Result Conundrum. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3744–3756.
[74] Silvan Reiner and Michael Grossniklaus. 2023. Sample-efficient cardinality estimation using geometric deep learning. *Proceedings of the VLDB Endowment* 17, 4 (2023), 740–752.
[75] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.
[76] Yufan Sheng, Xin Cao, Kaiqi Zhao, Yixiang Fang, Jianzhong Qi, Wenjie Zhang, and Christian S. Jensen. 2025. ACE: A Cardinality Estimator for Set-Valued Queries. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2112–2125.
[77] Charles Spearman. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology* 15, 1 (1904), 72–101. doi:10.2307/1412159
[78] Pavel Sulimov, Claude Lehmann, and Kurt Stockinger. 2025. GenJoin: Conditional Generative Plan-to-Plan Query Optimizer that Learns from Subplan Hints. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–25.
[79] Ji Sun and Guoliang Li. 2019. An end-to-end learning-based cost estimator. *Proceedings of the VLDB Endowment* 13, 3 (2019), 307–319.
[80] Ji Sun, Jintao Zhang, Zhaoyan Sun, Guoliang Li, and Nan Tang. 2021. Learned cardinality estimation: A design space exploration and a comparative evaluation. *Proceedings of the VLDB Endowment* 15, 1 (2021), 85–97.
[81] Jie Tan, Kangfei Zhao, Rui Li, Jeffrey Xu Yu, Chengzhi Piao, Hong Cheng, Helen Meng, Deli Zhao, and Yu Rong. 2025. Can large language models be query optimizer for relational databases? *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–28.
[82] Transaction Processing Performance Council (TPC). 2025. TPC-H Version 2 and Version 3. http://www.tpc.org/tpch/. Accessed: 2025.
[83] Kostas Tzoumas, Amol Deshpande, and Christian S Jensen. 2011. Lightweight graphical models for selectivity estimation without independence assumptions. *Proceedings of the VLDB Endowment* 4, 11 (2011), 852–863.

[84] Kapil Vaidya, Anshuman Dutt, Vivek Narasayya, and Surajit Chaudhuri. 2021. Leveraging query logs and machine learning for parametric query optimization. *Proceedings of the VLDB Endowment* 15, 3 (2021), 401–413.

[85] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[86] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

[87] Hanchen Wang, Ying Zhang, Lu Qin, Wei Wang, Wenjie Zhang, and Xuemin Lin. 2022. Reinforcement learning based query vertex ordering model for subgraph matching. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 245–258.

[88] Jiayi Wang, Chengliang Chai, Jiabin Liu, and Guoliang Li. 2021. FACE: A normalizing flow based cardinality estimator. *Proceedings of the VLDB Endowment* 15, 1 (2021), 72–84.

[89] Xiaoying Wang, Changbo Qu, Weiyuan Wu, Jiannan Wang, and Qingqing Zhou. 2021. Are we ready for learned cardinality estimation? *Proceedings of the VLDB Endowment* 14, 9 (2021), 1640–1654.

[90] Peizhi Wu and Gao Cong. 2021. A unified deep model of learning from both data and queries for cardinality estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 2009–2022.

[91] Peizhi Wu, Rong Kang, Tieying Zhang, Jianjun Chen, Ryan Marcus, and Zachary G. Ives. 2025. Data-Agnostic Cardinality Learning from Imperfect Workloads. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2519–2532.

[92] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: a new cardinality estimation framework for join queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[93] Ziniu Wu, Amir Shaikhha, Rong Zhu, Kai Zeng, Yuxing Han, and Jingren Zhou. 2020. BayesCard: Revitilizing bayesian frameworks for cardinality estimation. *arXiv preprint arXiv:2012.14743* (2020).

[94] Linglin Yang, Lei Zou, and Chunshan Zhao. 2025. NeuSO: Neural Optimizer for Subgraph Queries. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–28.

[95] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a query optimizer without expert demonstrations. In *Proceedings of the 2022 International Conference on Management of Data*. 931–944.

[96] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *Proceedings of the VLDB Endowment* 14, 1 (2020), 61–73.

[97] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Xi Chen, Pieter Abbeel, Joseph M Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep unsupervised cardinality estimation. *Proceedings of the VLDB Endowment* 13, 3 (2019), 279–292.

[98] Zhiming Yao, Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2025. A query optimization method utilizing large language models. *arXiv preprint arXiv:2503.06902* (2025).

[99] Junhao Ye, Jiahui Li, Lu Chen, Yuren Mao, Yunjun Gao, and Tianyi Li. 2024. LEAP: A Low-cost Spark SQL Query Optimizer using Pairwise Comparison. *Proceedings of the VLDB Endowment* 18, 3 (2024), 675–687.

[100] Xiang Yu, Chengliang Chai, Guoliang Li, and Jiabin Liu. 2022. Cost-based or learning-based? A hybrid query optimizer for query plan selection. *Proceedings of the VLDB Endowment* 15, 13 (2022), 3924–3936.

[101] Xiang Yu, Guoliang Li, Chengliang Chai, and Nan Tang. 2020. Reinforcement learning with tree-lstm for join order selection. In *2020 IEEE 36th international conference on data engineering (ICDE)*. IEEE, 1297–1308.

[102] Tianjing Zeng, Junwei Lan, Jiahong Ma, Wenqing Wei, Rong Zhu, Yingli Zhou, Pengfei Li, Bolin Ding, Defu Lian, Zhewei Wei, and Jingren Zhou. 2024. PRICE: A Pretrained Model for Cross-Database Cardinality Estimation. *Proceedings of the VLDB Endowment* 18, 3 (2024), 637–650.

[103] Yirui Zhan, Wen Nie, and Jun Gao. 2025. Sscard: Substring cardinality estimation using suffix tree-guided learned fm-index. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–24.

[104] Haozhe Zhang, Christoph Mayer, Mahmoud Abo Khamis, Dan Olteanu, and Dan Suciu. 2025. Lpbound: Pessimistic cardinality estimation using ℓ_p-norms of degree sequences. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.

[105] Kaixin Zhang, Hongzhi Wang, Yabin Lu, Ziqi Li, Chang Shu, Yu Yan, and Donghua Yang. 2024. Duet: efficient and scalable hybriD neUral rElation undersTanding. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 56–69.

[106] Kangfei Zhao, Jeffrey Xu Yu, Zongyan He, Rui Li, and Hao Zhang. 2022. Lightweight and accurate cardinality estimation by neural network gaussian process. In *Proceedings of the 2022 international conference on management of data*. 973–987.

[107] Yue Zhao, Gao Cong, Jiachen Shi, and Chunyan Miao. 2022. Queryformer: A tree transformer model for query plan representation. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1658–1670.

[108] Yue Zhao, Zhaodonghui Li, and Gao Cong. 2023. A comparative study and component analysis of query plan representation techniques in ml4db studies. *Proceedings of the VLDB Endowment* 17, 4 (2023), 823–835.

[109] Zhuoyue Zhao, Robert Christensen, Feifei Li, Xiao Hu, and Ke Yi. 2018. Random sampling over joins revisited. In *Proceedings of the 2018 International Conference on Management of Data*. 1525–1539.

[110] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1466–1479.

[111] Rong Zhu, Lianggui Weng, Bolin Ding, and Jingren Zhou. 2024. Learned query optimizer: what is new and what is next. In *Companion of the 2024 International Conference on Management of Data*. 561–569.

[112] Rong Zhu, Lianggui Weng, Wenqing Wei, Di Wu, Jiazhen Peng, Yifan Wang, Bolin Ding, Defu Lian, Bolong Zheng, and Jingren Zhou. 2024. PilotScope: Steering databases with machine learning drivers. *Proceedings of the VLDB Endowment* 17, 5 (2024), 980–993.

[113] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: fast, lightweight and accurate method for cardinality estimation. *Proceedings of the VLDB Endowment* 14, 9 (2021), 1489–1502.

[114] Yaoyu Zhu, Jintao Zhang, Guoliang Li, and Jianhua Feng. 2025. A Lightweight Learned Cardinality Estimation Model. *IEEE Transactions on Knowledge and Data Engineering* 37, 10 (2025), 5719–5734. doi:10.1109/TKDE.2025.3591025

A Detailed Explanation of Equation (1)

In this section, we first provide a detailed explanation of Equation (1), which is used to derive query cardinality estimates from a learned distribution over a full outer join. We then illustrate the main idea using a concrete example.

Recall that a group G consists of a set of tables $\mathcal{T}_G$ and their corresponding joins $\mathcal{J}_G$. The learned model captures the joint distribution of the full outer join $\bowtie G$. Consider a query Q whose referenced tables and joins are fully contained in G, i.e., $\mathcal{T}_Q \subseteq \mathcal{T}_G$ and $\mathcal{J}_Q \subseteq \mathcal{J}_G$. The goal is to estimate the query cardinality $\widehat{card}(Q)$ using the learned distribution over $\bowtie G$.

A direct evaluation of the query predicates on the full outer join distribution is not appropriate for two reasons. First, due to the null-padding semantics of full outer joins, a tuple in $\bowtie G$ does not necessarily correspond to a valid tuple from each table $T \in \mathcal{T}_Q$. Second, when $\mathcal{T}_G$ contains tables that are not referenced by Q, tuples that satisfy Q may appear multiple times in $\bowtie G$, with multiplicities induced by joins to unrelated tables. As a result, estimating $\widehat{card}(Q)$ from the full outer join distribution requires (i) excluding null-padded tuples that do not correspond to actual tuples in the queried tables, and (ii) correcting for join-induced tuple multiplicities.

These issues are addressed by augmenting the full outer join with two types of auxiliary columns (see Section 2.4). The indicator column I_T for each table $T \in \mathcal{T}_G$ specifies whether a tuple corresponds to an actual tuple in T or is null-padded. The fanout column $F_{T_1T_2}$ for each join $J = (T_1, T_2) \in \mathcal{J}_G$ records the number of matching tuples in T_2 for each join-key value in T_1. With these auxiliary columns, Equation (1) estimates the query cardinality as an expectation over the full outer join distribution. Specifically, the indicator $\mathbf{1}_{t[A]\in rng(A)}$ enforces the query predicates, the product of $t[I_T]$ terms excludes null-padded tuples from the queried tables, and the product of fanout terms $t[F_{T_1T_2}]$ in the denominator normalizes for tuple multiplicities induced by joins not in Q.

We now illustrate Equation (1) using an example. Figure 11 shows a "posts" table T_P, a "users" table T_U, and their full outer join augmented with the auxiliary columns. In this example, $G = \{T_P, T_U\}$, and the model is trained on $\bowtie G = T_P \bowtie T_U$. Consider the SQL query `SELECT COUNT(*) FROM users u WHERE u.u_date >= 2005`, which references only table T_U. The expectation in Equation (1) is evaluated over the four tuples in $\bowtie G$. Tuples that do not satisfy the predicate contribute zero through the indicator $\mathbf{1}_{t[A]\in rng(A)}$, while tuples corresponding to real user records contribute a value inversely proportional to their fanout. In this example, the four tuples contribute 0, $\frac{1}{2}$, $\frac{1}{2}$, and 1, respectively, where the factor $\frac{1}{2}$ arises from the fanout value $F_{U,P} = 2$. The expectation is therefore $(0 + \frac{1}{2} + \frac{1}{2} + 1)/4 = \frac{1}{2}$. Multiplying by $|\bowtie G| = 4$ yields the final estimate $\widehat{card}(Q) = 2$.

p_id	p_date	owner_u_id
1	2008	1
2	2009	2
3	2010	2

(a) Posts Table T_P

u_id	u_date
1	2004
2	2005
3	2006

(b) Users Table T_U

p_id	p_date	owner_u_id	u_id	u_date	I_P	I_U	$F_{P,U}$	$F_{U,P}$
1	2008	1	1	2004	1	1	1	1
2	2009	2	2	2005	1	1	1	2
3	2010	2	2	2005	1	1	1	2
null	null	null	3	2006	0	1	1	1

(c) Full Outer Join $T_P \bowtie T_U$

Fig. 11. Example tables and their full outer join with additional auxiliary columns.

p_id	p_date	owner_u_id
4	2025	2
5	2026	3

(a) Inserted Tuples to T_P

p_id	p_date	owner_u_id	u_id	u_date	I_P	I_U	$F_{P,U}$	$F_{U,P}$
1	2008	1	1	2004	1	1	1	1
2	2009	2	2	2005	1	1	1	3
3	2010	2	2	2005	1	1	1	3
4	2025	2	2	2005	1	1	1	3
5	2026	3	3	2006	1	1	1	1

(b) Updated Full Outer Join $T_P \bowtie T_U$

Fig. 12. Updates to the example table and the resulting full outer join.

B Updating Strategy of Correlated Grouping Methods

Correlated grouping methods, such as DeepDB and BayesCard, explicitly model the joint distribution over full outer joins of correlated tables. Supporting incremental updates for such methods requires determining how the modeled joint distribution changes under updates to the underlying base tables. Among existing estimators, only DeepDB provides an implementation of this update process. In the following, we analyze this strategy in detail and show that it relies on restrictive assumptions that limit its correctness and general applicability.

Consider a group $G = \{S, T\}$ and a timestamp t. At time t, the available data consist of the base tables $S_{\leq t}$ and $T_{\leq t}$, which are subsequently updated by ΔS and ΔT, respectively. In principle, an update evaluation should train a model on the initial join $S_{\leq t} \bowtie T_{\leq t}$ and then update it based on the induced join changes $\Delta(S \bowtie T)$, which should be derived solely from $(S_{\leq t}, T_{\leq t}, \Delta S, \Delta T)$.

However, the DeepDB implementation follows a different procedure, as discussed in Section 5.5. It first materializes the full outer join $S \bowtie T$ over the entire updated database. It then applies selection predicates of the form $\leq t$ on the temporal columns of the join result and treats the filtered tuples as the initial join $S_{\leq t} \bowtie T_{\leq t}$, while regarding the remaining tuples as the change set $\Delta(S \bowtie T)$.

This evaluation strategy exhibits three fundamental limitations. First, it implicitly relies on update information beyond the current timestamp when constructing $S_{\leq t} \bowtie T_{\leq t}$, whereas in a realistic online setting, only $S_{\leq t}$ and $T_{\leq t}$ are observable at time t. Second, filtering an updated full outer join does not, in general, yield the correct initial join $S_{\leq t} \bowtie T_{\leq t}$, and consequently may derive incorrect change sets, as we illustrate in the example below. Third, the strategy relies on explicit temporal attributes and update ordering assumptions, and therefore does not generalize to broader update scenarios that lack such temporal columns or exhibit out-of-order updates. In practice, $\Delta(S \bowtie T)$ should be derived from $(S_{\leq t}, T_{\leq t}, \Delta S, \Delta T)$ rather than inferred by applying temporal predicates to a precomputed full outer join. To address these limitations, we propose two practically feasible update strategies, Incremental Update and Parameter Refit, as discussed in Section 5.5.

We now illustrate all three update strategies using a concrete example. Continuing the example in Figure 11, Figure 12(a) shows two tuples inserted into table T_P, and Figure 12(b) shows the resulting full outer join $T_P \bowtie T_U$ after the update. Under the Incremental Update strategy, the change set $\Delta(T_P \bowtie T_U)$ is obtained by computing the difference between the initial join (Figure 11(c)) and the updated join (Figure 12(b)). In particular, the second to fourth tuples are modified due to the insertion of a post with `p_id = 4`, and a new tuple is introduced due to the insertion of a post with `p_id = 5`. Treating a tuple modification as one deletion followed by one insertion, $\Delta(T_P \bowtie T_U)$ consists of three deletions and four insertions. These changes are then applied to update the underlying model incrementally.

In this example, the total number of updates is seven, which exceeds the size of the updated full outer join (five tuples). This observation illustrates why the Parameter Refit strategy can be preferable in such cases. Instead of explicitly materializing $\Delta(T_P \bowtie T_U)$, this strategy recomputes the full outer join on the updated database and refits the parameters of the SPN or BN model while keeping the learned structure fixed.

Finally, the Temporal Filtering strategy, which is implemented in the DeepDB codebase, directly applies temporal predicates to the updated full outer join. In this example, the initial database in Figure 11 contains temporal values (the `p_date` and `u_date` columns) up to the year 2010. The strategy applies the predicates `p_date` ≤ 2010 and `u_date` ≤ 2010 to the updated join to derive the initial join, treating the remaining tuples as the change set. However, this procedure selects the first three rows in Figure 12(b) as the initial join, which differs from the correct initial join shown in Figure 11(c). In particular, the $F_{U,P}$ attributes in the second and third rows are set to 3 instead of 2,

encoding information about a future insertion of a post by the user with `u_id = 2`. Moreover, the last tuple in Figure 11(c) is omitted. This example demonstrates that the Temporal Filtering strategy may derive incorrect initial joins and change sets, as it violates the information available at time t.

C More Details of Training Workload Generation

To generate a SQL query, we first randomly select a subgraph from the join schema graph of database D to determine the join pattern, and then attach filtering predicates to it. The detailed procedure for generating training queries is shown in Algorithm 1.

Algorithm 1: Training Workload Query Generation

Input: A database $D = (\mathcal{T}_D, \mathcal{J}_D, \mathcal{A}_D)$
Output: A query $Q = (\mathcal{T}_Q, \mathcal{J}_Q, \mathcal{A}_Q, \mathcal{F}_Q)$ on D

1 $\mathcal{G}_D \leftarrow$ join schema graph with vertices $\mathcal{T}_D$ and edges $\mathcal{J}_D$
2 $n \sim \mathcal{U}(1, |\mathcal{T}_D|)$
3 $\mathcal{G}_{sub} \leftarrow$ connected subgraph of $\mathcal{G}_D$ of size n by random walk
4 $\mathcal{T}_Q \leftarrow V(\mathcal{G}_{sub})$
5 $\mathcal{J}_Q \leftarrow E(\mathcal{G}_{sub})$
6 $\mathcal{A}_Q \leftarrow \bigcup_{T \in \mathcal{T}_Q} \mathcal{A}_T$
7 $\mathcal{F}_Q \leftarrow \emptyset$
8 **foreach** $A \in \mathcal{A}_Q$ **do**
9 **with** *probability* p **do**
10 **if** A *is categorical* **then**
11 $op \leftarrow$ '='
12 **else** /* A *is numerical or string* */
13 $op \sim \{=, >, <, \geq, \leq, (>, <), (>, \leq), (\geq, <), (\geq, \leq)\}$
14 **if** op *is a one-sided operator* **then**
15 $v \sim \mathrm{dom}(A)$
16 $\mathcal{F}_Q \leftarrow \mathcal{F}_Q \cup \{(A, op, v)\}$
17 **else** /* op *is a two-sided operator* */
18 $v_1, v_2 \sim \mathrm{dom}(A)$
19 $\mathcal{F}_Q \leftarrow \mathcal{F}_Q \cup \{(A, op, \min(v_1, v_2), \max(v_1, v_2))\}$
20 **return** $Q(\mathcal{T}_Q, \mathcal{J}_Q, \mathcal{A}_Q, \mathcal{F}_Q)$

For a given database D, we first construct a schema graph based on its schema, where each node represents a table $T \in \mathcal{T}_D$ and each edge represents a join relationship $J \in \mathcal{J}_D$ between two tables (line 1). We then sample a random integer $n \in [1, |\mathcal{T}_D|]$ as the number of tables involved in the query (line 2). Starting from a randomly selected table, we perform a random walk on the schema graph to obtain a join query containing n tables (line 3). The resulting subgraph is treated as the query join graph, from which we derive the query tables $\mathcal{T}_Q$, query joins $\mathcal{J}_Q$, and the set of attributes $\mathcal{A}_Q$ associated with $\mathcal{T}_Q$ (lines 4–6).

Lines 7–19 describe the generation of filtering predicates. Each attribute in $\mathcal{A}_Q$ is considered with a certain probability for predicate generation. For categorical attributes, we generate an equality predicate (=). For numerical and string attributes, we generate either a comparison predicate (>, <, >=, or <=) or a range predicate formed by combining two comparison predicates. The literal values

used in these predicates are randomly sampled from the corresponding attribute domains in the database.

Finally, each generated SQL query is executed in PostgreSQL to obtain its ground-truth cardinality. Queries with zero cardinality or those whose execution time exceeds a predefined timeout of 500 seconds are discarded.

D Configuration of PostgreSQL Extended Statistics

PostgreSQL allows extended statistics [70] to be constructed over multiple columns within a single table. We use a heuristic to identify correlated column groups on which to construct extended statistics.

For each table, we consider columns that can appear in filtering predicates and compute the pairwise randomized dependence coefficient (RDC) [51] among them. We consider a subset of columns to be correlated if its corresponding correlation graph is connected, where each node represents a column and an edge connects two columns whose RDC exceeds a threshold θ.

To control the construction overhead, we restrict each subset to at most six columns and, for each subset size, retain at most 200 subsets with the highest average pairwise RDC. We set $\theta = 0.3$ in our experiments.

E Details of Database Snapshot Construction Based on Temporal Columns

We describe the procedure for constructing a database snapshot based on temporal attributes, which enables the evaluation of dynamic updates by inserting tuples that arrive after the snapshot.

Given a timestamp t, we first filter each table using its associated temporal attribute. We then enforce PK–FK constraints to ensure referential integrity, resulting in an initial database instance.

In the IMDb dataset, the `title` table contains the temporal attribute `production_year`, which is used to filter tuples with respect to timestamp t. The remaining five tables in the JOB-LIGHT workload each contain a foreign key `movie_id` referencing `title.id`. We retain only those tuples whose `movie_id` values have matching entries in the filtered `title` table, thereby enforcing PK–FK constraints across the schema. This process yields a consistent database snapshot at time t. Similarly, in the STATS dataset, all tables except `tag` contain a temporal attribute, either `date` or `creationdate`. We apply the same procedure by first performing temporal filtering on each table, followed by enforcing PK–FK constraints to remove tuples that violate referential integrity.

We exclude ErgastF1 because the `pitstops` table only has records from 2011 onward. This snapshot already contains roughly 73% of the full database, leaving only a small fraction for update evaluation. Earlier timestamps produce an empty `pitstops` table and incomplete schema, which prevents training the initial model.

F Additional Experimental Results

F.1 Training Data Efficiency of Query-Driven Methods

Table 6 reports the execution time of query plans generated using cardinality estimates produced by learned estimators trained with different numbers of queries, aggregated over the entire workload. Overall, the execution time decreases as more training queries become available, exhibiting trends consistent with the Q-Error and P-Error results.

F.2 Generalization Under Query Workload Drift

Figure 13 presents detailed results on P-Error and total workload execution time under the three training configurations, complementing the Q-Error analysis in the main text.

Table 6. Execution time (in seconds) with different numbers of training queries. PostgreSQL, PostgreSQL-Ext, LpBound, and PRICE (zero-shot prediction) do not rely on training queries and are reported only for comparison.

Workload	Method	# of training queries					
		500	1000	5000	10000	50000	100000
JOB-LIGHT	MSCN	1568	1070	1066	1068	994	1142
	RobustMSCN	1218	1647	1217	1097	1045	990
	JGMP	1102	1004	1044	1001	980	995
	ALECE	1056	1022	1022	993	952	962
	PRICE-FT	1180	1025	1009	1009	1343	1170
	PostgreSQL	1138	1138	1138	1138	1138	1138
	PostgreSQL-Ext	1127	1127	1127	1127	1127	1127
	LpBound	996	996	996	996	996	996
	PRICE	1030	1030	1030	1030	1030	1030
STATS-CEB	MSCN	3986	4015	4344	8156	3664	3293
	RobustMSCN	4558	3861	3238	3025	3037	3313
	JGMP	3567	3034	3045	3171	3117	3104
	ALECE	3707	3126	3687	3161	3189	3239
	PRICE-FT	5797	5938	3828	3684	7075	4736
	PostgreSQL	7068	7068	7068	7068	7068	7068
	PostgreSQL-Ext	7056	7056	7056	7056	7056	7056
	LpBound	3011	3011	3011	3011	3011	3011
	PRICE	6887	6887	6887	6887	6887	6887
ErgastF1	MSCN	8507	7612	7318	7368	13154	6618
	RobustMSCN	10059	11563	7477	6333	6735	6880
	JGMP	7602	8371	6431	6434	6182	6207
	ALECE	7021	7051	6600	6611	6950	6822
	PRICE-FT	7544	7537	7446	7488	6857	6937
	PostgreSQL	6961	6961	6961	6961	6961	6961
	PostgreSQL-Ext	7000	7000	7000	7000	7000	7000
	LpBound	6252	6252	6252	6252	6252	6252
	PRICE	6934	6934	6934	6934	6934	6934

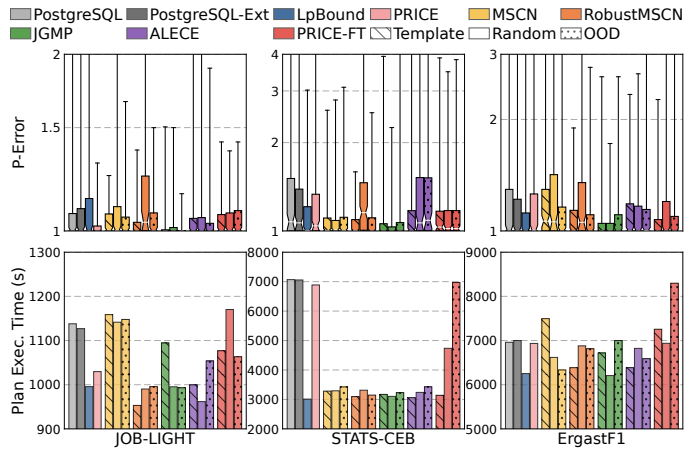


Fig. 13. P-Error and total workload execution time under different training configurations.

P-Error follows trends similar to those of Q-Error across all configurations, with Template generally yielding the lowest errors. Notably, ALECE is particularly sensitive to workload shifts. On

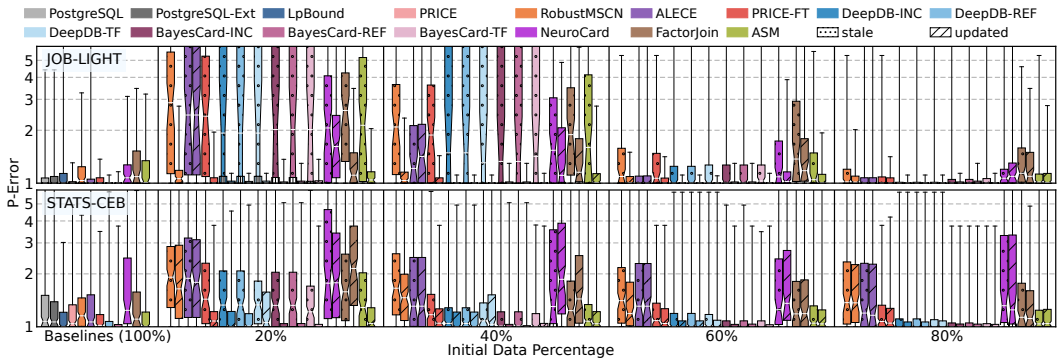


Fig. 14. P-Error distributions under varying initial data ratios for the stale model (dotted hatching), the updated model (diagonal hatching), and the retrained fresh model (the leftmost "Baselines" without hatching).

STATS-CEB, the P-Error of ALECE under Random and OOD is comparable to that of PostgreSQL. ALECE outperforms PostgreSQL only in the Template setting. This limited generalization can be attributed to ALECE's architecture, which encodes query features into a single flat vector, constraining its ability to generalize to unseen workloads.

In terms of total workload execution time, Template and Random generally outperform OOD. However, we observe notable discrepancies between P-Error distributions and execution time, as a small number of long-running queries can dominate total runtime. This observation is consistent with the findings in Section 4.3. For example, on STATS-CEB, PRICE-FT exhibits similar P-Error under Random and Template, yet incurs substantially higher execution time under Random. Further analysis shows that Random leads to a severely suboptimal plan for a single complex query (Q122), resulting in an execution time of 2396.4 seconds, compared to 649.1 seconds under Template, due to large estimation errors for this query.

F.3 Dynamic Updates

This section presents additional experimental results on dynamic updates, focusing on P-Error and execution time.

Figure 14 illustrates the P-Error results. Overall, most learned estimators adapt effectively to data updates, as evidenced by the reduced P-Error of the updated models compared to the stale ones. However, some updated models still exhibit noticeable degradation relative to freshly retrained models, consistent with the Q-Error observations in Section 5.5. Among query-driven methods, PRICE-FT exhibits the smallest performance degradation, achieving P-Error comparable to the model retrained on the full database.

For data-driven methods, all variants of DeepDB and BayesCard show minimal performance degradation, achieving P-Errors comparable to those of the retrained models and obtaining the lowest P-Error among data-driven estimators. On STATS-CEB, DeepDB exhibits greater performance degradation as the initial data percentage decreases and the update magnitude increases, whereas BayesCard remains relatively robust to larger updates. This difference may stem from the independence assumptions introduced in the product nodes of SPNs in DeepDB, which may not hold well after updates, whereas the BNs in BayesCard only assume causal relationships between attributes and are therefore more robust under data updates.

The execution time of plans derived from cardinality estimates produced by stale models, updated models, and freshly retrained models is reported in Table 7. We limit the total workload execution

Table 7. Execution time (in seconds) of different methods under varying initial data ratios. TLE denotes Time Limit Exceeded.

Method	JOB-LIGHT									STATS-CEB								
	Initial Data Ratio																	
	20%		40%		60%		80%		100%	20%		40%		60%		80%		100%
	Stale	Updated	Stale	Updated	Stale	Updated	Stale	Updated	Fresh	Stale	Updated	Stale	Updated	Stale	Updated	Stale	Updated	Fresh
RobustMSCN	3340	1169	2235	1091	1266	1079	1021	1012	990	3184	3733	3347	3418	3318	5401	3601	3477	3313
ALECE	2246	2197	1853	1627	1014	1037	967	1022	962	3580	3708	3677	3718	3483	3605	3431	3533	3239
PRICE-FT	3455	1156	3305	1054	1296	1047	1301	1045	1170	3710	7183	3379	7157	3135	7162	7153	7132	4736
DeepDB-INC	14478	1012	14141	997	2490	1004	2224	1008	972	3311	3416	3340	3154	3229	3153	3093	3139	3081
DeepDB-REF	14214	1029	14140	1011	2503	1002	2200	1005	972	3478	3402	3456	3151	3431	3171	3130	3169	3081
DeepDB-TF	14499	1012	14423	997	2556	1001	2259	1004	972	3589	3327	3584	3253	3374	3108	3169	3142	3081
BayesCard-INC	TLE	1022	2942	1047	1228	1021	1523	1026	980	3306	3235	3338	3242	3108	3173	3064	3196	3002
BayesCard-REF	TLE	996	2935	1006	1197	989	1481	1002	980	3298	3135	3284	3135	3083	3001	3121	2979	3002
BayesCard-TF	TLE	1028	2954	1014	1215	1029	1519	1030	980	3478	3240	3300	3251	3094	3127	3095	3032	3002
NeuroCard	2068	1149	1772	1128	1232	1144	1190	1093	1084	3695	3557	3503	3649	3637	3713	3635	3721	3276
FactorJoin	1249	1056	1161	1155	1130	1084	1023	1070	1027	11081	4559	11033	4449	10737	11296	4090	4343	3756
ASM	2334	1180	2243	1166	1267	1072	1265	1067	1088	4206	3285	3447	3246	3335	3280	3223	3273	2988
PostgreSQL	–	–	–	–	–	–	–	–	1138	–	–	–	–	–	–	–	–	7068
PostgreSQL-Ext	–	–	–	–	–	–	–	–	1127	–	–	–	–	–	–	–	–	7056
LpBound	–	–	–	–	–	–	–	–	996	–	–	–	–	–	–	–	–	3011
PRICE	–	–	–	–	–	–	–	–	1030	–	–	–	–	–	–	–	–	6887
TrueCard	–	–	–	–	–	–	–	–	931	–	–	–	–	–	–	–	–	2958

Table 8. Q-Error percentiles of PostgreSQL, ASM, and ASM-Hybrid on JOB-LIGHT, STATS-CEB, and ErgastF1.

Workload	Method	Q-Error				
		50%	80%	90%	95%	99%
JOB-LIGHT	PostgreSQL	1.390	3.635	9.032	22.260	169.406
	ASM	1.963	7.425	16.350	27.992	54.999
	ASM-Hybrid	1.968	7.480	15.932	26.169	54.999
STATS-CEB	PostgreSQL	1.575	6.783	18.752	50.518	964.598
	ASM	1.703	4.372	12.090	88.091	9846.559
	ASM-Hybrid	1.756	4.702	13.588	87.068	9039.307
ErgastF1	PostgreSQL	1.213	2.766	6.511	17.772	83.664
	ASM	1.883	15.294	57.000	138.902	784.176
	ASM-Hybrid	1.643	10.206	35.710	107.380	706.182

time to five hours and executions that do not finish within this limit are reported as Time Limit Exceeded (TLE). The effectiveness of model updates is reflected in the reduced workload runtime compared to stale models. DeepDB and BayesCard achieve the best execution times and show performance comparable to the retrained models, indicating that traditional ML models such as SPNs and BNs are more amenable to data updates.

F.4 Integration with Traditional Methods

We further investigate hybrid approaches for data-driven methods using ASM as a case study. In Section 4.1, we observe that ASM exhibits severe underestimation on ErgastF1, which is associated with sampling failures [40, 65]. A natural idea is therefore to detect such failures and fall back to a traditional estimator.

ASM draws samples from per-table DAR models, and a full sample s contains a sampled tuple s_T for each table $T \in \mathcal{T}_Q$. We regard s as invalid if any s_T does not correspond to an actual tuple in T. If more than 99% of the samples are invalid, we fall back to PostgreSQL's estimate. We refer to this estimator as ASM-Hybrid. Table 8 reports the Q-Error percentiles of ASM-Hybrid. On ErgastF1, ASM-Hybrid improves over ASM across the reported percentiles, suggesting that the fallback strategy can partially mitigate the impact of sampling failures. The differences on JOB-LIGHT and STATS-CEB are minor, consistent with sampling failures being much less severe on these workloads (Figure 2).

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026